ARTIFICIAL INTELLIGENCE

FOLLOW THIS SYLLABUS AND THE NOTES FOR YOUR JUT UNIVERSITY EXAM.

Course Outcomes:

CO1: To understand the basic concepts, history, and applications of Artificial Intelligence and its key areas.
CO2: To apply supervised learning algorithms for solving classification and regression problems.
CO3: To implement unsupervised learning techniques for clustering and dimensionality reduction.
CO4: To analyze and evaluate the performance of different AI and ML models using appropriate metrics.
CO5: To comprehend the principles of reinforcement learning and apply them to simple problems.
CO6: To apply knowledge of AI and ML to develop intelligent systems for real-world applications

Course Contents:

UNIT I

AI – History of AI – Agents – Structure of Intelligent Agents – Environments – Problem Solving Methods – Problem Solving Agents – Formulating Problems – Search Strategies – Breadth-First – Uniform Cost – Depth-First – Depth-Limited – Bidirectional– Informed Search – Best-First Heuristic Functions – Memory Bounded Search – A* – SMA* – Iterative Improvement Algorithms – Hill Climbing – Simulated Annealing – Measure of Performance and Analysis of Search Algorithms.

UNIT II

Game Playing – Perfect Decisions – Imperfect Decisions – Alpha-Beta Pruning – Knowledge Based Agent – Wumpus World Environment – Propositional Logic – Agent for Wumpus World – First Order Logic – Syntax – Semantics – Extensions – Using First Order Logic – Representation Change in the World – Goal Based Agents.

UNIT III

Knowledge Base – Knowledge Representation – Production Based System – Frame Based System – Inference – Backward Chaining – Forward Chaining.

UNIT IV

Learning from Agents – Inductive Learning – Types of Machine Learning – Supervised Learning – Learning Decision Trees – Support Vector Machines – Neural and Belief Networks – Perceptron – Multi-layer Feed Forward Networks – Bayesian Belief Networks.

UNIT V

Unsupervised Learning – K-means Clustering – Hierarchical Clustering – Agglomerative and Divisive Clustering – Fuzzy Clustering.

UNIT I: Introduction to AI, Intelligent Agents and Search

1. Artificial Intelligence (AI) — Introduction

Artificial Intelligence (AI) is the branch of computer science that deals with building machines and software capable of performing tasks that would normally require human intelligence. Such tasks include understanding natural language, recognising objects and sounds, learning from experience, solving problems, planning, and making decisions under uncertainty. In simple words, AI tries to make a computer 'think' and 'act' the way an intelligent human would — but it does this using algorithms, data structures, mathematics and logic instead of a biological brain.

A commonly used definition, given by Elaine Rich, is: 'Artificial Intelligence is the study of how to make computers do things which, at the moment, people do better.' AI systems are generally judged along two dimensions — thinking vs acting, and humanly vs rationally — which gives us four categories of definitions:

- Systems that think like humans (cognitive modelling approach).
- Systems that act like humans (the Turing Test approach).
- Systems that think rationally (the 'laws of thought' / logic approach).
- Systems that act rationally (the rational agent approach — the modern, most widely accepted view).

Example: A chess-playing program such as AlphaZero is an AI system because it perceives the state of the chessboard, reasons about possible moves, and chooses the action (move) that is most likely to lead to victory — a task that needs 'intelligence' when performed by a human.

1.1 History of AI

The history of AI can be understood as a series of waves of optimism and 'AI winters' (periods of reduced funding and interest):

- 1943 — McCulloch and Pitts proposed the first mathematical model of an artificial neuron, laying the foundation of neural networks.
- 1950 — Alan Turing published 'Computing Machinery and Intelligence' and proposed the Turing Test to judge machine intelligence.
- 1956 — The Dartmouth Conference, organised by John McCarthy, Marvin Minsky, Claude Shannon and others, is regarded as the official birth of AI as a field; the term 'Artificial Intelligence' was coined here.
- 1956–1974 — The 'golden years': early programs like the Logic Theorist and General Problem Solver were developed; expectations were very high.
- 1974–1980 — First AI Winter: progress slowed because of limited computing power and unrealistic promises, so funding was cut.
- 1980–1987 — Rise of Expert Systems (e.g., MYCIN, DENDRAL) that used rule-based knowledge to solve domain-specific problems; commercial interest returned.
- 1987–1993 — Second AI Winter caused by the collapse of the specialised Lisp-machine market and the limitations of expert systems.
- 1993–2011 — AI became more mathematical, with strong growth in machine learning, probabilistic reasoning (Bayesian networks), and intelligent agents.

- 2011–present — The Deep Learning era: with the availability of big data, GPUs, and better algorithms, AI achieved breakthroughs in image recognition, speech recognition, game playing (AlphaGo), and natural language processing (large language models).

Exam Tip: Questions on 'History of AI' are commonly asked for 2–3 marks. Remember the Dartmouth Conference (1956) as the official birth year, and the two AI winters.

2. Agents

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators. This is one of the most fundamental concepts in AI because almost every AI system (a robot, a chatbot, a game-playing program, a thermostat) can be modelled as an agent.

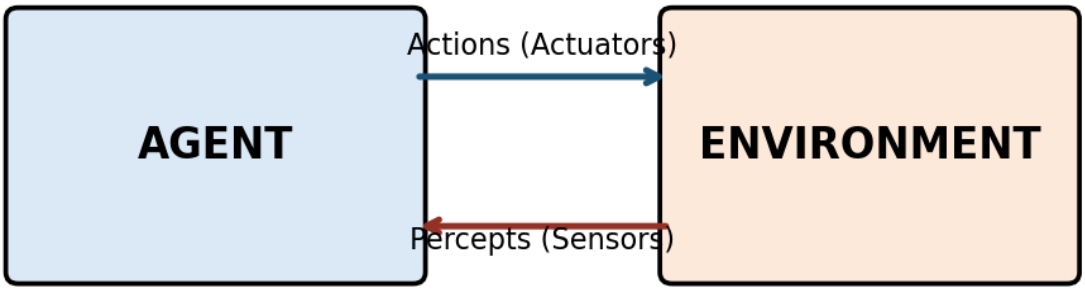


Fig 2.1: The Agent interacts with its Environment through sensors (percepts) and actuators (actions).

For example, a human agent has eyes, ears, and other organs as sensors, and hands, legs, and vocal tract as actuators. A robotic agent might have cameras and infrared range finders as sensors, and motors as actuators. A software agent receives file contents, keystrokes, or network packets as percepts, and acts by displaying output on the screen, writing files, or sending network packets.

2.1 Percept, Percept Sequence and Agent Function

- Percept: the agent's perceptual input at any given instant.
- Percept Sequence: the complete history of everything the agent has perceived so far.
- Agent Function: a mathematical function that maps every possible percept sequence to an action. It describes the agent's behaviour abstractly.
- Agent Program: the concrete implementation of the agent function, running on some physical architecture (hardware).

Example: In a vacuum-cleaner world with two rooms (A and B), the percept is a pair such as [location, status] — e.g., [A, Dirty]. The agent function could specify: if the current square is dirty, then suck; else move to the other square. This simple rule already demonstrates how an agent function converts percepts into actions.

2.2 Rational Agent

A rational agent is one that does the 'right thing' — it selects the action that maximises its performance measure, given the percept sequence it has received so far and whatever built-in knowledge it has. Rationality depends on four things: the performance measure that defines success, the agent's prior knowledge of the environment, the actions the agent can perform, and the agent's percept sequence so far.

Note that rationality is not the same as omniscience or perfection — a rational agent can still make mistakes if the outcome is genuinely unpredictable, but it must act reasonably given the information available to it.

3. Structure of Intelligent Agents

The job of AI is to design the agent program — the function that implements the agent's behaviour and maps percepts to actions — which runs on some architecture (a computer with sensors and actuators). Based on how much 'intelligence' or internal structure they use, agents can be classified into five main types.

3.1 Simple Reflex Agents

A simple reflex agent selects actions based only on the current percept, ignoring the rest of the percept history. It works using condition-action rules of the form 'if condition then action' (also called situation-action rules). These agents are simple and fast, but they only work correctly if the environment is fully observable — i.e., the correct decision can be made based on the current percept alone.

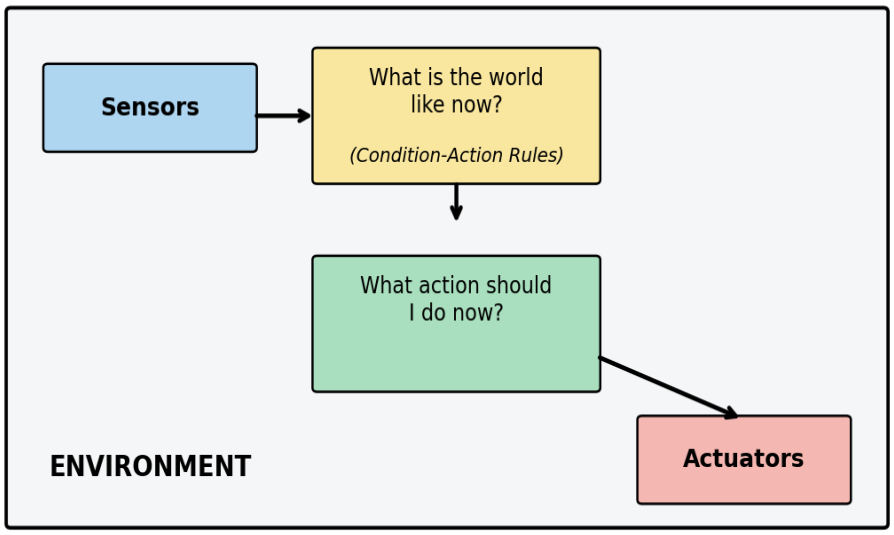


Fig 3.1: Structure of a Simple Reflex Agent using condition-action rules.

Example: A thermostat is a simple reflex agent — if the temperature falls below the set point, turn the heater ON; if it rises above the set point, turn it OFF. It does not remember past temperatures; it only reacts to the present percept.

3.2 Model-Based Reflex Agents

A model-based reflex agent keeps track of the part of the world it cannot currently see by maintaining an internal state. This internal state is updated over time using knowledge of how the world evolves (a 'transition model') and how the agent's own actions affect the world. This allows the agent to handle partial observability, which a simple reflex agent cannot.

Example: A self-driving car uses a model-based reflex agent approach — even when a pedestrian momentarily passes behind another vehicle (occluded from the camera), the internal model keeps track of where the pedestrian is likely to be.

3.3 Goal-Based Agents

A goal-based agent, besides maintaining a model of the world, also has a goal — a description of desirable situations. It chooses actions that will (eventually) achieve its goal, often by considering 'what will happen if I do action A?' and searching or planning over sequences of actions. This gives it flexibility because the goal can be changed without redesigning the whole agent.

Example: A GPS navigation system is goal-based — given the goal 'reach the destination', it searches through possible routes and selects the sequence of turns that achieves the goal, and it can easily be given a new goal (a different destination).

3.4 Utility-Based Agents

Goals alone are not always enough to generate high-quality behaviour, because there may be many ways to achieve a goal, some better than others (faster, safer, cheaper). A utility-based agent uses a utility function that maps a state (or a sequence of states) to a real number, indicating the degree of 'happiness' or desirability of that state. The agent chooses the action that maximises expected utility, allowing rational decisions even when goals conflict or are uncertain.

Example: A ride-sharing app's route-planning algorithm is utility-based — it does not just find 'a' route to the destination (the goal) but the one that best balances time, fuel cost, and tolls (utility).

3.5 Learning Agents

A learning agent has the ability to improve its performance over time through experience. It has four conceptual components: the Performance Element (which selects actions, similar to the earlier agent types), the Learning Element (responsible for making improvements), the Critic (which gives feedback on how well the agent is doing, using a fixed performance standard), and the Problem Generator (which suggests exploratory actions that may not be optimal in the short term but lead to better long-term learning).

Example: A spam-email filter is a learning agent — the performance element classifies emails as spam or not spam, and the learning element updates its rules whenever the user marks a wrongly-classified email, so the filter's accuracy keeps improving.

4. Environments

The environment is everything that surrounds the agent and with which the agent interacts. Before designing an agent, it is essential to understand the properties of its task environment (often abbreviated as the PEAS description: Performance measure, Environment, Actuators, Sensors). Environments can be classified along several dimensions:

- Fully Observable vs Partially Observable: In a fully observable environment, the agent's sensors give it access to the complete state of the environment at each point in time (e.g., chess). In a partially observable environment, sensors give only noisy or incomplete information (e.g., a self-driving car cannot see around a blind corner).

- Deterministic vs Stochastic: In a deterministic environment, the next state is completely determined by the current state and the agent's action (e.g., a puzzle like the 8-puzzle). In a stochastic environment, there is randomness / uncertainty in outcomes (e.g., dice games, taxi driving with unpredictable traffic).
- Episodic vs Sequential: In an episodic environment, the agent's experience is divided into atomic 'episodes' where the choice of action in each episode depends only on the episode itself (e.g., defect-detection on an assembly line). In a sequential environment, the current decision can affect all future decisions (e.g., chess, where every move affects the rest of the game).
- Static vs Dynamic: A static environment does not change while the agent is deliberating (e.g., crossword puzzles). A dynamic environment can change during the agent's decision-making (e.g., driving a car, where other vehicles keep moving).
- Discrete vs Continuous: A discrete environment has a finite/countable number of distinct states and actions (e.g., chess board positions). A continuous environment has states and actions that vary smoothly (e.g., robot arm angles, taxi driving speed).
- Single-Agent vs Multi-Agent: A single-agent environment has only one agent acting (e.g., solving a crossword alone). A multi-agent environment has multiple agents that may cooperate or compete (e.g., chess is a two-agent, competitive environment).
- Known vs Unknown: This refers to the agent's (or designer's) state of knowledge about the 'rules' of the environment, not the state of the environment itself.

Example (PEAS for a self-driving taxi): Performance measure — safety, speed, legal driving, comfort, profit; Environment — roads, traffic, pedestrians, weather; Actuators — steering, accelerator, brake, horn, display; Sensors — cameras, GPS, speedometer, sonar, engine sensors. The taxi environment is partially observable, stochastic, sequential, dynamic, continuous, and multi-agent.

5. Problem Solving Methods and Problem-Solving Agents

When the correct action to take is not obvious from a simple lookup or a rule, an intelligent agent must plan ahead by considering sequences of actions that lead to desirable states — this is called problem solving through search. A problem-solving agent is a kind of goal-based agent that decides what to do by finding a sequence of actions that leads from the initial state to a goal state.

The problem-solving agent follows a fixed sequence of four phases:

1. Goal Formulation: based on the current situation, the agent decides what it wants to achieve.
2. Problem Formulation: the agent decides what actions and states to consider given the goal (abstracting away irrelevant detail).
3. Search: the agent simulates sequences of actions in its model to find a path from the initial state to the goal — this produces a solution.
4. Execution: the agent carries out the actions recommended by the solution, one at a time.

Example: Consider a tourist in Chandigarh who wants to reach Delhi. Goal formulation: 'Be in Delhi.' Problem formulation: states are cities, actions are drives between adjacent cities. Search: the agent examines route options (via a map) to find a sequence of cities forming a path to Delhi. Execution: the tourist actually drives the chosen route.

6. Formulating Problems

Formally, a search problem can be defined using five components:

- Initial State: the state the agent starts in.
- Actions: a description of the possible actions available to the agent in a given state, ACTIONS(s).
- Transition Model: a description of what each action does; RESULT(s, a) returns the state that results from performing action a in state s.
- Goal Test: determines whether a given state is a goal state.
- Path Cost: a function that assigns a numeric cost to each path; usually the sum of the costs of individual actions along the path.

The initial state, actions and transition model together implicitly define the state space — the set of all states reachable from the initial state by any sequence of actions. This state space can be visualised as a directed graph in which nodes are states and edges are actions/transitions. A path in this graph is a sequence of states connected by actions. A solution to the problem is a path from the initial state to a goal state, and an optimal solution is the one with the lowest path cost among all solutions.

Example — The 8-puzzle: States describe the arrangement of 8 numbered tiles and a blank on a 3x3 board; Initial state is the given (scrambled) arrangement; Actions move the blank Left/Right/Up/Down; Transition model gives the resulting arrangement; Goal test checks whether the arrangement matches the target configuration; Path cost is usually the number of moves (each move costs 1).

7. Search Strategies

A search strategy is defined by picking the order in which nodes are expanded. Search strategies are broadly divided into two categories:

- Uninformed (Blind) Search: has no additional information about states beyond that provided in the problem definition. These strategies can only generate successors and distinguish a goal state from a non-goal state. Examples: BFS, Uniform Cost Search, DFS, Depth-Limited Search, Iterative Deepening, Bidirectional Search.
- Informed (Heuristic) Search: uses problem-specific extra knowledge (a heuristic function) beyond the problem definition, which helps find solutions more efficiently. Examples: Best-First Search, A*, SMA*, Hill Climbing, Simulated Annealing.

The performance of a search strategy is evaluated on four criteria, which is exactly the 'Measure of Performance and Analysis of Search Algorithms' topic covered later in this unit: Completeness, Optimality, Time Complexity, and Space Complexity.

7.1 Breadth-First Search (BFS)

Breadth-First Search expands the shallowest (closest to the root) unexpanded node first. It is implemented using a FIFO (First-In-First-Out) queue for the frontier — newly generated nodes (children) go to the back of the queue, so shallower nodes are expanded before deeper nodes. BFS explores the search tree level by level, like ripples spreading outward from the initial state.

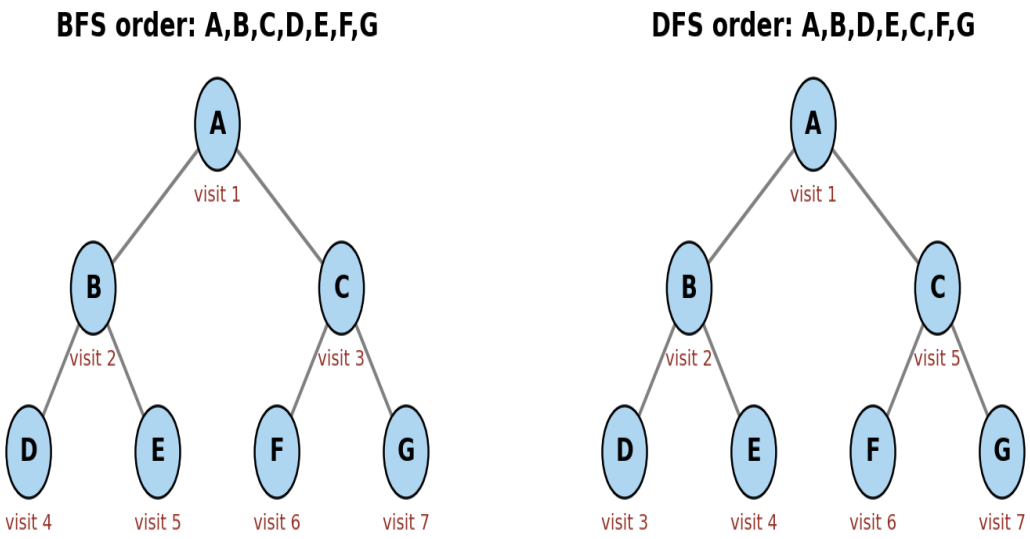


Fig 7.1: BFS visits nodes level by level (left) while DFS dives deep along one branch first (right).

Properties: BFS is complete (it will find a solution if one exists, provided the branching factor is finite) and it is optimal if all step costs are equal, because it always finds the shallowest goal node first. However, its time and space complexity are both O(b^d), where b is the branching factor and d is the depth of the shallowest solution — this exponential space requirement is BFS's biggest drawback, as the frontier can become extremely large.

Example: Finding the minimum number of moves to solve a puzzle, or finding the shortest friend-chain between two people in a social network (all edges have equal 'cost'), is naturally solved with BFS.

7.2 Uniform-Cost Search (UCS)

Uniform-Cost Search is used when step costs are not all identical. Instead of expanding the shallowest node, UCS always expands the node with the lowest path cost g(n) from the initial state, using a priority queue ordered by path cost. UCS is a generalisation of BFS (BFS is the special case where every step costs 1).

UCS is complete and optimal, provided step costs are all greater than or equal to some small positive constant. Its time and space complexity is O(b^(1+floor(C*/ε))), where C* is the cost of the optimal solution and ε is the minimum step cost — which can be much worse than b^d if there are many small steps.

Example: Finding the cheapest flight route between two cities, where each flight leg has a different ticket price, is solved with UCS rather than plain BFS.

7.3 Depth-First Search (DFS)

Depth-First Search always expands the deepest unexpanded node in the current frontier first. It is implemented using a LIFO (Last-In-First-Out) stack (or simply by recursion). The search dives down one branch of the tree as far as possible; when it reaches a dead end (or the maximum depth) it backtracks to the nearest unexplored branch.

DFS has modest memory requirements — it only needs to store the path from the root to the current node, plus the unexpanded sibling nodes, giving a space complexity of O(bm), where m is the maximum depth of the tree — much better than BFS's O(b^d). However, DFS is not complete in infinite state spaces (it can get stuck going

down an infinitely deep or looping path), and it is not optimal, because it may find a longer / costlier solution before a shorter one. Its worst-case time complexity is O(b^m).

7.4 Depth-Limited Search

Depth-Limited Search solves DFS's problem of infinite paths by imposing a fixed depth limit, l — nodes at depth l are treated as if they have no successors. This guarantees termination, so it resolves the infinite-path problem, but it introduces incompleteness if the shallowest goal is actually deeper than the chosen limit l. Choosing a good limit l requires prior knowledge of the problem (e.g., the diameter of the state space).

7.5 Iterative Deepening Search

Iterative Deepening Search combines the benefits of BFS and DFS: it repeatedly runs Depth-Limited Search with increasing depth limits (0, 1, 2, 3, ...) until a goal is found. It is complete and optimal (for equal step costs) like BFS, but keeps DFS's modest linear memory requirement, O(bd). Although it seems wasteful to regenerate shallow nodes repeatedly, the overhead is small because most nodes in a search tree lie near the bottom, so the extra re-generation of upper-level nodes is relatively cheap.

7.6 Bidirectional Search

Bidirectional Search runs two simultaneous searches — one forward from the initial state, and one backward from the goal state — and stops when the two frontiers meet in the middle. The motivation is that b^(d/2) + b^(d/2) is much smaller than b^d, so bidirectional search can dramatically reduce time and space requirements. Its main requirement is that the goal state (or states) must be explicitly known, and the actions must be reversible (or an inverse-actions definition must exist) so the backward search is possible.

Example: Finding the shortest route between two cities on a large road map is often done bidirectionally — searching outward from both the source city and the destination city simultaneously.

8. Informed (Heuristic) Search

Informed search strategies use problem-specific knowledge, encoded in a heuristic function h(n), to guide the search more efficiently towards the goal, rather than blindly exploring in all directions.

8.1 Heuristic Functions

A heuristic function h(n) estimates the cost of the cheapest path from node n to a goal node. It is problem-specific: for example, in route-finding problems, the straight-line distance to the goal is a natural heuristic; in the 8-puzzle, the number of misplaced tiles, or the sum of Manhattan distances of tiles from their goal positions, are common heuristics.

A heuristic is called admissible if it never overestimates the true cost to reach the goal — i.e., h(n) <= h*(n), where h*(n) is the actual optimal cost from n to the goal. A heuristic is called consistent (or monotonic) if, for every node n and every successor n' generated by action a, h(n) <= c(n,a,n') + h(n'), i.e., the estimated cost does not decrease by more than the actual step cost. Every consistent heuristic is also admissible.

8.2 Best-First Search

Best-First Search is a general approach in which a node is selected for expansion based on an evaluation function f(n); the node with the lowest f(n) value is chosen first (implemented with a priority queue). Different choices of f(n) give different specific algorithms:

- Greedy Best-First Search: uses f(n) = h(n) only, always expanding the node that appears closest to the goal. It is fast, but not complete (can get stuck in loops) and not optimal, because it ignores the cost already spent to reach n.
- A* Search: uses f(n) = g(n) + h(n), where g(n) is the actual cost so far to reach n, and h(n) is the estimated cost from n to the goal. This is the most widely used informed search algorithm.

8.3 A* Search

A* Search evaluates nodes using f(n) = g(n) + h(n): g(n) is the exact cost from the start to node n, and h(n) is the heuristic estimate of the cost from n to the nearest goal. Because f(n) estimates the total cost of the cheapest solution through n, A* expands the node that is likely to be on the cheapest overall path.

Key result: A* using Tree-Search is optimal if h(n) is admissible; A* using Graph-Search is optimal if h(n) is consistent. Because f-values are non-decreasing along any path (when h is consistent), A* is guaranteed to find the optimal solution. However, A*'s memory usage is still exponential in the worst case, since it keeps all generated nodes in memory.

Example: A* is used in GPS navigation software and video-game pathfinding, where straight-line distance to the destination serves as h(n), guiding the search efficiently while still finding the shortest actual route.

8.4 Memory-Bounded Search and SMA*

Because A* can run out of memory on large problems, memory-bounded variants were developed:

- Iterative-Deepening A* (IDA*): performs iterative deepening using f(n) = g(n)+h(n) as the cutoff instead of depth, keeping memory usage linear.
- Recursive Best-First Search (RBFS): mimics best-first search using only linear space, by keeping track of the f-value of the best alternative path available from any ancestor of the current node.
- Simplified Memory-Bounded A* (SMA*): SMA* behaves like A* until memory is full. When memory runs out, it drops the leaf node with the highest f-value (the worst node) from the frontier to free up space, and 'backs up' the forgotten value into its parent, so that the parent remembers the best value of its forgotten children. This lets SMA* make full use of whatever memory is available, and it is complete if there is enough memory to store the shallowest solution path, and optimal if enough memory is available to store the best solution path.

> **Exam Tip:** SMA* is a favourite viva/theory question: remember it 'forgets the worst node and remembers its value at the parent' when memory is full — this is the key differentiator from plain A*.

9. Iterative Improvement Algorithms

In many optimisation problems (e.g., the 8-queens problem, VLSI layout, job-shop scheduling), the path to the goal is irrelevant — only the final goal state itself matters. In such cases, local search / iterative improvement algorithms are used. These algorithms operate using a single current state (rather than multiple paths) and move only to neighbouring states, gradually improving that one state. They use very little memory (usually constant, since only the current state and a few neighbours need to be stored), and they can often find reasonable solutions in large or even infinite (continuous) state spaces where systematic search algorithms are impractical.

9.1 Hill Climbing

Hill Climbing is a simple iterative improvement algorithm that continually moves in the direction of increasing value (or decreasing cost) — that is, it looks at the neighbouring states of the current state and moves to whichever neighbour has the best (highest) evaluation, stopping when no neighbour has a higher value than the current state. It is often described as 'greedy local search' because it always grabs the immediately best neighbour without looking ahead, and it does not maintain a search tree — only the current node and its immediate neighbours are recorded.

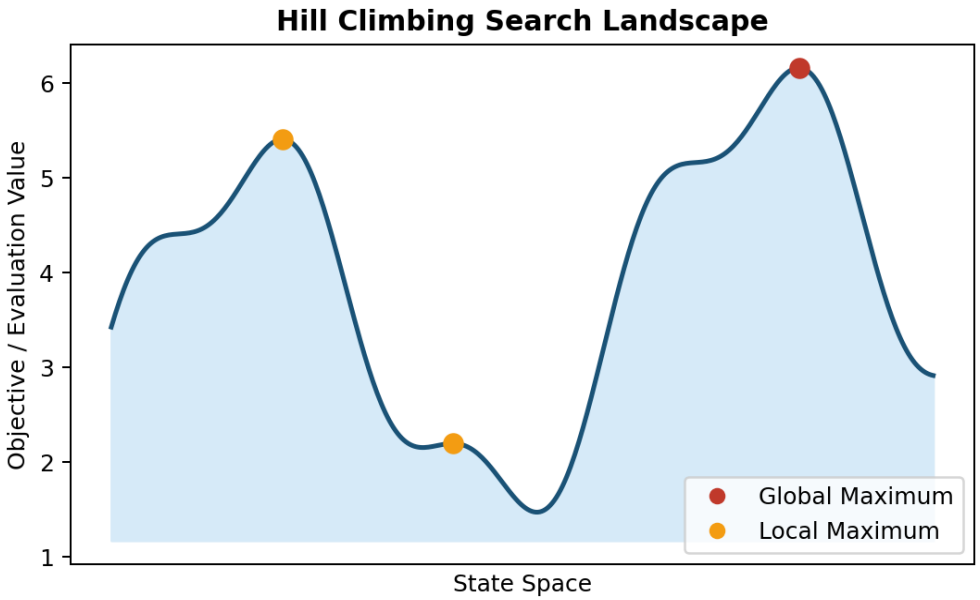


Fig 9.1: Hill Climbing can get trapped in a local maximum instead of reaching the global maximum.

Hill climbing frequently gets stuck for the following reasons:

- Local Maxima: a peak that is higher than all its neighbours but lower than the global maximum — once here, hill climbing cannot escape because all neighbouring moves make things worse.
- Plateaus: a flat area of the state-space landscape where all neighbouring states have the same value, so the algorithm has no direction to move in and effectively gets stuck wandering.
- Ridges: a sequence of local maxima that is very difficult to navigate because the orientation of the ridge compared to the available moves makes progress slow, even though the overall trend leads upward.

Variants used to mitigate these problems include Stochastic Hill Climbing (chooses randomly among better neighbours, weighted by improvement), First-Choice Hill Climbing (generates neighbours randomly until one is found that is better than the current state), and Random-Restart Hill Climbing (runs hill climbing multiple times from randomly generated initial states and keeps the best result).

Example: In the 8-queens problem, hill climbing starts with a random arrangement of 8 queens on the board and repeatedly moves one queen to the position within its column that most reduces the number of attacking pairs, until no single move can reduce it further.

9.2 Simulated Annealing

Simulated Annealing is inspired by the metallurgical process of annealing, in which metals are heated to a high temperature and then slowly cooled to reach a low-energy, stable crystalline state. Instead of picking only the best move (as in hill climbing), simulated annealing picks a random move; if the move improves the situation, it

is always accepted, but if the move makes things worse, it is accepted with a probability that decreases exponentially with the 'badness' of the move (ΔE) and decreases over time as a control parameter T (the 'temperature') is gradually lowered according to a cooling schedule. The acceptance probability is typically given as p = e^(ΔE / T).

At high temperature, the algorithm behaves almost like a random walk (lots of exploration, escaping local maxima easily); as T decreases towards zero, the algorithm behaves more and more like hill climbing (mostly exploiting good moves). It can be proven that if T is lowered slowly enough, simulated annealing will find the global optimum with probability approaching 1. This is why simulated annealing is widely used for large combinatorial optimisation problems such as VLSI layout, scheduling and the travelling-salesman problem.

10. Measure of Performance and Analysis of Search Algorithms

Search strategies are compared and evaluated on the basis of four standard criteria:

- Completeness: Is the algorithm guaranteed to find a solution when one exists?
- Optimality: Does the strategy find the optimal (least-cost) solution among all possible solutions?
- Time Complexity: How long does it take to find a solution, usually measured as the number of nodes generated/expanded?
- Space Complexity: How much memory is required to perform the search, usually measured as the maximum number of nodes stored in memory at any time?

Time and space complexity are usually expressed in terms of three quantities that characterise the difficulty of the problem: the branching factor b (maximum number of successors of any node), the depth d of the shallowest goal node, and the maximum length m of any path in the state space.

Criterion	BFS	UCS	DFS	Depth-Limited	Iterative Deepening	A*
Complete?	Yes	Yes	No	No	Yes	Yes (admissible
Optimal?	Yes (equal cost)	Yes	No	No	Yes (equal cost)	Yes
Time	O(b^d)	O(b^(1+C*/ε))	O(b^m)	O(b^l)	O(b^d)	Exponential (bounded by h)
Space	O(b^d)	O(b^(1+C*/ε))	O(bm)	O(bl)	O(bd)	Exponential

Note: b = branching factor, d = depth of shallowest solution, m = maximum depth of the state space, l = depth limit, C* = cost of the optimal solution, ε = smallest step cost.

Exam Tip: This comparison table is an excellent quick-revision tool and is often directly asked as a university question — memorise the complexity column carefully.

UNIT II: Game Playing and Knowledge Representation using Logic

1. Game Playing

Games have historically been an important application area for AI because they provide a well-defined, competitive environment that is easy to formalise, yet extremely hard to solve completely — for example, chess has roughly 10^40 legal positions. Games are typically modelled as a search problem involving two players (usually called MAX and MIN) who move alternately, and where each player tries to maximise their own outcome while minimising the opponent's.

A game can be formally defined using the following components: the Initial State (board setup and whose turn it is), a function TO-MOVE(state) that indicates which player moves, ACTIONS(state) giving the legal moves, RESULT(state, move) the transition model, IS-TERMINAL(state) that checks whether the game has ended, and UTILITY(state, player) which gives a numeric value for the outcome of a terminal state (for example, +1 for a win, -1 for a loss, 0 for a draw in chess).

1.1 The Minimax Algorithm — Perfect Decisions

When a full search of the game tree down to terminal states is computationally feasible, a game-playing agent can make a perfect decision using the Minimax algorithm. The idea is: MAX tries to choose the move that leads to the state with the highest utility value, while MIN tries to choose the move that leads to the state with the lowest utility value (assuming MIN plays optimally against MAX). The minimax value of a node is computed recursively:

- If the node is terminal, minimax(n) = UTILITY(n).
- If it is MAX's turn, minimax(n) = the maximum of minimax(s) over all successors s of n.
- If it is MIN's turn, minimax(n) = the minimum of minimax(s) over all successors s of n.

This is called making 'Perfect Decisions' because, assuming the opponent also plays optimally, minimax guarantees the best possible outcome for MAX. The main drawback is that a full minimax search has time complexity O(b^m), which is completely impractical for games like chess (branching factor around 35, depth around 80).

1.2 Imperfect, Real-Time Decisions

Because it is usually infeasible to search all the way to terminal states in complex games, real programs make imperfect decisions by cutting off the search early — this is called a Heuristic (evaluation) cutoff. Instead of expanding all the way to a terminal state, the search is stopped at some depth limit, and a heuristic evaluation function EVAL(state) estimates the utility of the state as if it were terminal (e.g., material advantage in chess). This turns the exact minimax value into an approximate one, trading optimality for feasibility, which is why such decisions are called 'imperfect' — the agent cannot guarantee it has chosen the truly best move, only the best move according to its limited lookahead and heuristic estimate.

Additional practical techniques include Quiescence Search (extending the search at 'unstable' positions, such as right after a capture, so the evaluation is not misleading) and Forward Pruning (discarding moves that appear clearly bad without fully evaluating them, to save time).

1.3 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimisation of the Minimax algorithm that eliminates the need to explore branches of the game tree that cannot possibly influence the final decision — it produces exactly the same decision as minimax while examining far fewer nodes. It maintains two values as it searches: alpha (α) — the best value that MAX can guarantee so far along the current path, and beta (β) — the best value that MIN can guarantee so far along the current path. Whenever the algorithm finds that a node's value would make α >= β at some ancestor, it can safely stop (prune) exploring the remaining children of that node, because a rational opponent would never allow that branch to be reached.

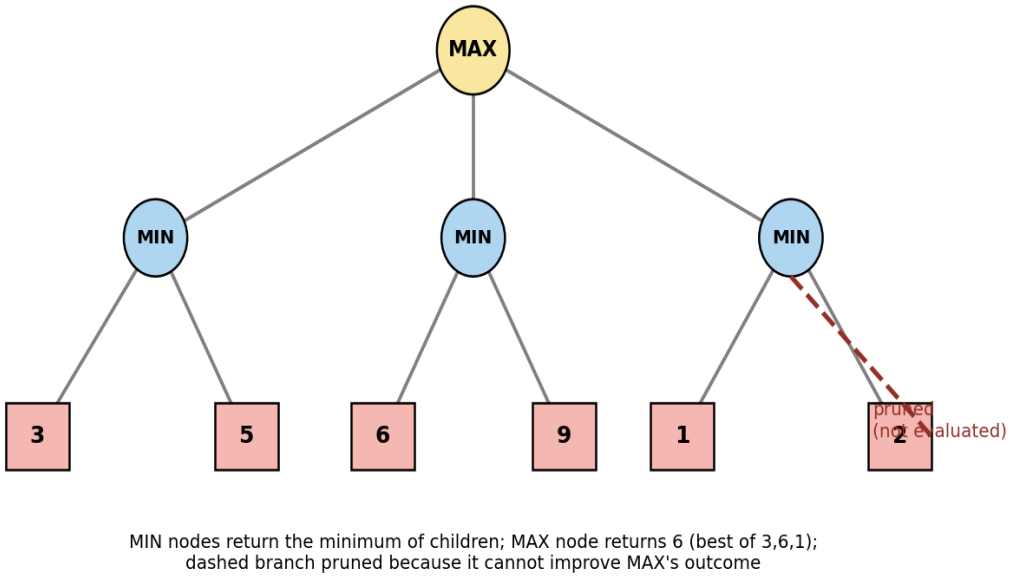


Fig 1.1: Alpha-Beta Pruning avoids evaluating branches (dashed) that cannot affect the final MAX decision.

In the best case (when moves happen to be ordered so that the best move is examined first), alpha-beta pruning reduces the time complexity from O(b^m) to O(b^(m/2)) — effectively doubling the depth that can be searched in the same amount of time compared to plain minimax. This is why alpha-beta pruning, rather than plain minimax, is used in essentially every practical game-playing program.

Worked example: In Figure 1.1, the MAX node has three MIN children. The first MIN child evaluates its two leaves (3 and 5) and returns 3. The second MIN child evaluates its first leaf (6); since MAX already has a guaranteed 3, and this branch already offers 6 (>=3, so it could still help) — after seeing the second leaf (9) the branch returns 6. The third MIN child evaluates its first leaf (1); since MAX already has a value of 6 from the second branch, and this branch already can only give at most 1 so far (which is worse for MAX than the guaranteed 6), the second leaf under this branch can be pruned — there is no need to evaluate it because it cannot change MAX's final decision of 6.

Exam Tip: Alpha-beta pruning always gives the same answer as minimax; only the amount of computation changes — this line ('same result, less work') is a favourite exam distinction.

2. Knowledge-Based Agents

A knowledge-based agent is one whose central component is a Knowledge Base (KB) — a set of sentences expressed in some formal representation language (typically a form of logic) that represents facts about the world that the agent believes to be true. The agent operates using two main functions: TELL, which adds new sentences (percepts, facts) into the KB, and ASK, which queries the KB to determine what action to take, and the agent decides on actions by performing logical inference over its KB — deriving new sentences from old ones using rules of logic.

At an abstract level, a knowledge-based agent can be described in three steps for every time step: (1) it TELLs the KB what it has perceived, (2) it ASKs the KB what action it should perform (using logical reasoning combined with the goal), and (3) it TELLs the KB which action was chosen, and then performs it. This design lets the agent be built by simply adding sentences to the KB, at what is called the 'knowledge level' — describing what the agent knows, independent of how it is implemented.

The advantages of the knowledge-based approach include: the agent can be told new facts directly (learning does not require reprogramming), the agent can combine facts to reach new conclusions it was never explicitly told, and the agent's behaviour is transparent and can be explained by showing the chain of logical deductions.

3. The Wumpus World Environment

The Wumpus World is a classic cave-exploration puzzle used as a testbed to illustrate the use of a knowledge-based agent and logical reasoning. The agent is placed in a grid of rooms (typically 4x4) representing a cave. Somewhere in the cave lurks the Wumpus, a deadly beast that will eat any agent that enters its room; there are also bottomless pits scattered through the cave, and a heap of gold in one room.

A Typical Wumpus World (4x4 Grid)

OK			PIT
Stench	Wumpus	Breeze	
Breeze	OK	Breeze Gold?	
START OK	Breeze	PIT	Breeze

Fig 3.1: A sample Wumpus World — percepts like 'Breeze' and 'Stench' warn the agent about nearby pits and the Wumpus.

PEAS description of the Wumpus World:

- Performance measure: +1000 for climbing out of the cave with the gold, -1000 for being eaten by the Wumpus or falling into a pit, -1 for each action taken, and -10 for using the single available arrow.
- Environment: a 4x4 grid of rooms; the agent starts in the room at bottom-left [1,1], facing east; the locations of the Wumpus, pits, and gold are randomly assigned to other squares.
- Actuators: the agent can Move Forward, Turn Left/Turn Right by 90°, Grab (an object in the current room), Shoot (the single arrow in the direction it is facing), and Climb (out of the cave when at [1,1]).

- Sensors: the agent has five percepts, given as a 5-tuple — Stench (in the room and rooms directly adjacent to the Wumpus), Breeze (in rooms directly adjacent to a pit), Glitter (in the room containing gold), Bump (when the agent walks into a wall), and Scream (heard everywhere when the Wumpus is killed by the arrow).

This environment has the following properties: it is discrete, static, single-agent, and partially observable (the agent can only sense its current square — it cannot see pits or the Wumpus directly unless right next to it). It is essentially deterministic, except that the initial locations of the Wumpus, pits and gold are chosen randomly at the start.

Example of reasoning: If the agent perceives a Breeze in square [1,2], it can logically infer that there must be a pit in one of the adjacent squares ([1,3] or [2,2]), even though it cannot see the pit directly. If it later also perceives no Breeze in a room adjacent to [2,2], it can logically deduce that the pit must be in [1,3], not [2,2] — this kind of deduction from percepts to conclusions about hidden aspects of the world is exactly what propositional and first-order logic (discussed next) allow the agent to do formally.

4. Propositional Logic

Propositional Logic (also called Boolean logic) is the simplest logic that illustrates the basic concepts of logical representation and reasoning. It deals with propositions — declarative statements that are either true or false — combined using logical connectives.

4.1 Syntax of Propositional Logic

The syntax of propositional logic defines the allowable sentences. The atomic sentences consist of a single proposition symbol, each of which stands for a fact that can be true or false (e.g., P1,1 could mean 'there is a pit in square [1,1]'). Two special proposition symbols, True and False, are also allowed. Complex sentences are constructed from simpler sentences using five logical connectives:

- ¬ (Negation / NOT): ¬P means 'P is false'.
- ∧ (Conjunction / AND): P ∧ Q means 'both P and Q are true'.
- ∨ (Disjunction / OR): P ∨ Q means 'at least one of P or Q is true'.
- ⇒ (Implication / IF-THEN): P ⇒ Q means 'if P is true then Q must also be true'; P is called the premise/ antecedent, Q the conclusion/consequent.
- ⇔ (Biconditional / IFF): P ⇔ Q means 'P is true exactly when Q is true' (both directions of implication hold).

4.2 Semantics of Propositional Logic

Semantics defines the rule for determining the truth of a sentence with respect to a particular model — an assignment of true/false values to every proposition symbol. Once we know the truth values of the atomic propositions, the truth of any complex sentence can be evaluated recursively using standard truth tables for ¬, ∧, ∨, ⇒, ⇔. For example, if P is true and Q is false, then P ∧ Q is false, but P ∨ Q is true, and P ⇒ Q is false.

Important semantic notions: A sentence is valid (a tautology) if it is true in every model (e.g., P ∨ ¬P). A sentence is satisfiable if it is true in at least one model. A sentence is unsatisfiable if it is false in every model (e.g., P ∧ ¬P). One sentence α entails another sentence β (written α ⊨ β) if, in every model where α is true, β is also true — entailment is the formal notion that underlies logical inference.

Example (Wumpus World in propositional logic): Let B1,1 mean 'there is a breeze in square [1,1]', and P1,2, P2,1 mean 'there is a pit in [1,2] / [2,1]' respectively. The rule of the game can be written as: B1,1 ⇔ (P1,2 ∨ P2,1). If the agent perceives B1,1 to be true, it can combine this sentence with logical inference rules to conclude facts about where pits might be.

5. Agent for the Wumpus World

A logical, knowledge-based agent for the Wumpus World maintains a knowledge base containing (a) the fixed rules of the game (e.g., the breeze/pit and stench/wumpus adjacency rules), and (b) the percepts it has received at every visited square, each tagged with the time and location. At every step, before choosing an action, the agent uses logical inference (such as model checking or resolution) to ASK the KB questions like 'is square [2,2] safe to enter?' If the KB entails that a square is safe (no pit, no Wumpus), the agent can plan a path there; if the KB entails that a square is dangerous, the agent avoids it; and if the KB cannot determine either way, the agent must either explore cautiously or make a calculated risk assessment.

This approach demonstrates the power of the knowledge-based agent design: by encoding just the rules of adjacency (Breeze ⇔ neighbouring Pit, Stench ⇔ neighbouring Wumpus) and combining them with actual observed percepts, the agent can safely navigate the cave, find the gold, and return, using pure logical deduction rather than guessing — even though it never directly perceives the pits or the Wumpus except through its percepts.

6. First-Order Logic (FOL)

Propositional logic is limited because it cannot conveniently express general facts about objects, their properties, and the relationships between them (for example, it needs a completely separate proposition for each pit in each square, with no way to say 'all pits cause a breeze in adjacent squares' as a single general rule). First-Order Logic (FOL, also called Predicate Logic) overcomes this limitation by introducing objects, relations (predicates), functions, and quantifiers, giving it much greater expressive power.

6.1 Syntax of First-Order Logic

The building blocks of FOL syntax are:

- Constants: name specific objects, e.g., John, 2, Delhi.
- Predicates: represent relations or properties of objects and evaluate to true or false, e.g., Brother(John, Richard), King(John).
- Functions: map objects to other objects (not true/false), e.g., FatherOf(John).
- Variables: stand for unspecified objects, e.g., x, y.
- Connectives: the same five as propositional logic — ¬, ∧, ∨, ⇒, ⇔.
- Quantifiers: Universal Quantifier ∀ ('for all') and Existential Quantifier ∃ ('there exists').
- Terms: expressions that refer to an object — either a constant, a variable, or a function applied to terms.
- Atomic sentence: a predicate (relation) applied to a list of terms, e.g., Brother(Richard, John).

Quantifiers deserve special attention: ∀x P(x) means P(x) is true for every object x in the domain of discourse (e.g., ∀x King(x) ⇒ Person(x), 'every king is a person'). ∃x P(x) means there exists at least one object x for which

P(x) is true (e.g., ∃x Crown(x) ∧ OnHead(x, John), 'there is a crown on John's head'). A common mistake to avoid: universal quantification typically uses ⇒ as the main connective, while existential quantification typically uses ∧.

6.2 Semantics of First-Order Logic

The semantics of FOL are defined with respect to a model that consists of a domain of objects and an interpretation that maps constant symbols to objects, predicate symbols to relations over those objects, and function symbols to functions over those objects. A sentence is true in a model under a particular interpretation if the relations/functions/objects it refers to actually hold according to that mapping. As with propositional logic, we can define entailment, validity, and satisfiability, but now with respect to first-order models — which, unlike propositional models, can have infinitely many objects, making FOL considerably more expressive but also making inference more challenging (FOL is only semi-decidable in general).

6.3 Extensions of First-Order Logic

Basic FOL can be extended for greater expressiveness or convenience:

- Equality (=): allows stating that two terms refer to the same object, e.g., FatherOf(John) = Richard.
- Higher-order logic: allows quantification over predicates and functions themselves, not just over objects (e.g., 'there exists a property that both John and Mary share').
- Modal / Temporal logics: extend FOL with operators for necessity/possibility or for time (e.g., 'it is always true that...', 'eventually...'), useful for reasoning about belief, knowledge, and time-varying facts.
- Fuzzy logic / Probabilistic extensions: allow degrees of truth or probabilities instead of strict true/false, useful for reasoning under uncertainty.

6.4 Using First-Order Logic — Representation Change in the World

FOL is used to represent knowledge about a domain by writing a set of axioms (general rules) plus specific facts. As new information is perceived, the knowledge base must be updated to reflect the change in the world — this is called representation of change. There are two common approaches:

- Situation Calculus: every predicate/function that can change over time is given an extra argument representing the 'situation' (a snapshot of the world at a point in time), e.g., At(Agent, [1,1], S0) means the agent is at [1,1] in situation S0. Actions transform one situation into another via a function Result(action, situation), allowing the KB to reason about how the world changes as actions are performed.
- Fluents: predicates or functions whose value can change over time are called fluents (as opposed to eternal, unchanging facts, which are simply predicates without a situation argument). For example, Alive(Wumpus, s) is a fluent since the Wumpus may be alive in one situation and dead in a later one.

Example: In the Wumpus World, using situation calculus we might write: Holding(Gold, s) ⇒ Holding(Gold, Result(TurnRight, s)) — turning right does not cause the agent to drop the gold, i.e., the Holding fluent persists across that particular action. Such rules (called 'frame axioms') are needed to specify what does not change when an action is performed, in addition to rules (called 'effect axioms') specifying what does change.

7. Goal-Based Agents (in the context of Logic and Planning)

As introduced in Unit I, a goal-based agent chooses its actions by considering which action sequences will lead to states satisfying its goal. When combined with a logical knowledge base (as in the Wumpus World agent above), a goal-based agent can use logical inference not only to determine which states are currently safe/true, but also

to reason about hypothetical future states resulting from candidate action sequences, effectively merging logical reasoning with search/planning to decide on the best course of action towards its goal (for example, 'go to the square containing the gold, grab it, and return to the start').

This connects Unit I (agents, search) with Unit II (logic): search provides the mechanism to explore possible action sequences, while logic (propositional or first-order) provides the mechanism to represent what is known about the world and to soundly infer new facts — together, they allow a goal-based, knowledge-based agent to operate intelligently and safely in a partially observable environment such as the Wumpus World.

UNIT III: Knowledge Representation and Inference

1. Knowledge Base

A Knowledge Base (KB) is the central component of any knowledge-based system — it is an organised collection of facts, rules, and relationships about a particular domain, represented in a form that a computer program can store, retrieve, and reason over. The KB is populated using domain knowledge supplied by human experts or extracted from data, and it is used by an Inference Engine to derive new conclusions or answer queries.

A good knowledge base should satisfy several important properties: Representational Adequacy (the ability to represent all the kinds of knowledge needed for the domain), Inferential Adequacy (the ability to derive new knowledge from existing knowledge), Inferential Efficiency (the ability to direct inference in productive directions using control information), and Acquisitional Efficiency (the ability to acquire new knowledge easily, whether through automated learning or human input).

Example: An expert system for medical diagnosis, such as MYCIN, has a knowledge base containing facts like 'the organism is gram-positive' and rules like 'IF the infection is primary-bacteremia AND the site of culture is one of the sterile sites AND the suspected portal of entry is the gastrointestinal tract, THEN there is suggestive evidence that the identity of the organism is bacteroides.' The inference engine combines such rules with patient-specific facts to suggest a diagnosis.

2. Knowledge Representation (KR)

Knowledge Representation is the sub-field of AI concerned with how knowledge about the world can be represented and how reasoning processes can use that knowledge to derive new information or make decisions. Since the way knowledge is represented directly affects how efficiently and effectively a program can reason, choosing an appropriate representation scheme is one of the most important design decisions in building an intelligent system.

Broadly, knowledge can be classified into several types, each of which may need a different kind of representation:

- Declarative Knowledge: simple statements of facts about the world (e.g., 'the sky is blue'), typically represented using logic or semantic networks.
- Procedural Knowledge: knowledge about how to do something, i.e., a sequence of actions or a method (e.g., how to solve a quadratic equation), typically represented using production rules or algorithms.
- Meta-Knowledge: knowledge about knowledge itself (e.g., which rules are more reliable, or in what order to try different reasoning strategies).
- Heuristic Knowledge: rule-of-thumb knowledge based on experience that is useful but not guaranteed to be correct (e.g., 'if traffic is heavy on the highway, take the service road').
- Structural Knowledge: knowledge about how concepts relate to and are structured with respect to one another (e.g., a car has an engine, wheels, and a chassis).

Common knowledge representation schemes used in AI include Logic (propositional and first-order, covered in Unit II), Semantic Networks (graphs where nodes represent objects/concepts and labelled edges represent relationships, e.g., 'IS-A', 'HAS-A'), Production (Rule-Based) Systems, and Frame-Based Systems — the latter two are discussed in detail below.

3. Production-Based System (Rule-Based System)

A Production System (also called a rule-based system) is one of the most widely used architectures for representing procedural / heuristic knowledge in AI. It consists of three main components:

- A Global Database (Working Memory): holds the current facts or state of the problem-solving process, which is continuously read from and written to.
- A Set of Production Rules (the Rule Base): each rule is expressed as a condition-action (IF-THEN) pair — IF a specified pattern of conditions is satisfied by the current contents of the global database, THEN the corresponding action is performed (which usually updates the global database in some way).
- A Control Strategy (Inference / Recognise-Act Cycle): decides which applicable rule to fire when several rules' conditions are satisfied simultaneously (this is called conflict resolution), and manages the overall cycle of matching rules against the database and executing them.

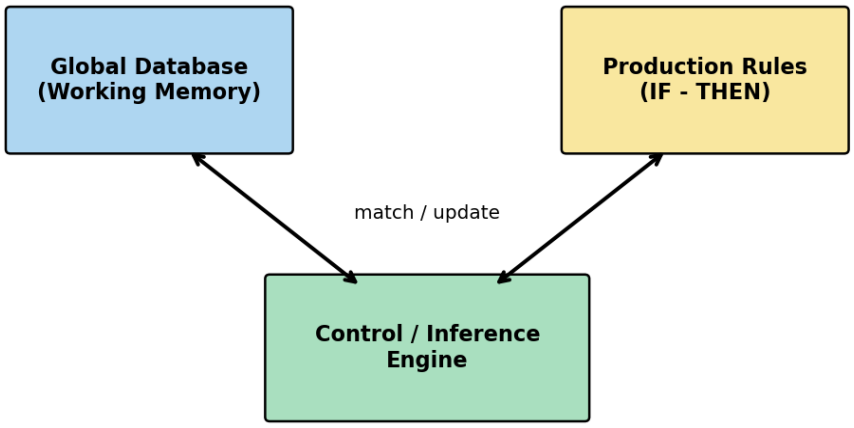


Fig 3.1: Architecture of a Production System — the control/inference engine repeatedly matches rules against the working memory and fires applicable ones.

The basic operation of a production system follows a repeated 'Match–Select–Act' (recognise-act) cycle: (1) Match — compare the rules' IF-parts (conditions) against the current contents of the global database to find the set of rules whose conditions are satisfied (called the conflict set); (2) Select (Conflict Resolution) — choose one rule from the conflict set to fire, using a strategy such as rule priority, specificity (most specific rule wins), or recency (rule matching the most recently added fact wins); (3) Act — execute the THEN-part of the selected rule, which typically adds, deletes, or modifies facts in the global database. This cycle repeats until no more rules match, or a goal condition is reached.

Example: A simple production system for animal identification might contain rules such as: 'IF the animal has fur AND gives milk THEN it is a mammal'; 'IF the animal is a mammal AND has stripes THEN it is a tiger.' Given the facts 'has fur' and 'gives milk' in the global database, the first rule fires, adding 'is a mammal' to the database; this newly added fact then allows the second rule (if 'has stripes' is also known) to fire, concluding 'is a tiger'.

Advantages of production systems include modularity (each rule is an independent chunk of knowledge that can be added or removed without directly editing other rules) and a natural mapping to human expert reasoning ('if situation X, then do Y'), which makes them easy to build and understand. Their main disadvantages are that, as

the rule base grows large, it becomes hard to guarantee there are no conflicting or redundant rules, and the matching process can become computationally expensive.

4. Frame-Based System

A Frame is a data structure used to represent a stereotyped situation, object, or concept, along with all the knowledge associated with it, in one organised unit — similar in spirit to a 'record' or 'object' in programming, but specifically designed for AI knowledge representation. Frame-based representation was proposed by Marvin Minsky as a way to capture structured, real-world knowledge more naturally than flat logical sentences.

A frame consists of a collection of Slots, and each slot holds a Value (which can itself be another frame, a number, a string, a default value, a procedure to compute the value when needed — called a 'procedural attachment' or 'demon' — or a pointer/link to a related frame). For example, a frame representing the concept 'Car' might have slots such as: Number-of-Wheels (default value 4), Fuel-Type, Manufacturer, Owner (a link to a 'Person' frame), and so on.

Frames are usually organised into a hierarchy using IS-A links, which allows Inheritance — a powerful feature where a specific frame automatically 'inherits' the slots and default values of its more general parent frame, unless it explicitly overrides them with its own value. This closely mirrors object-oriented programming concepts such as classes, instances, and inheritance.

Example: Consider a frame hierarchy: 'Vehicle' (a general frame, with slot Number-of-Wheels = unknown) → 'Car' (IS-A Vehicle, overrides Number-of-Wheels = 4) → 'MyHondaCity' (IS-A Car, an instance with Owner = 'Ramesh', Colour = 'White'). The instance 'MyHondaCity' automatically inherits Number-of-Wheels = 4 from the Car frame without it needing to be stated explicitly, and it can further specialise with its own specific slot values such as Colour.

Frames are particularly effective for representing structured, real-world knowledge with natural defaults and exceptions (e.g., 'birds typically fly, but a penguin frame can override the Flies slot to False'), and they group all related knowledge about a concept together, making retrieval efficient. Their main limitation is that reasoning with frames (especially with inheritance exceptions and procedural attachments) can be less formally rigorous than logic-based systems, and complex frame hierarchies can become difficult to maintain.

5. Inference

Inference is the process by which new facts or conclusions are logically derived from existing facts and rules stored in the knowledge base. It is the mechanism that allows a knowledge-based system to go beyond what it was explicitly told, and answer questions or make decisions that require combining multiple pieces of knowledge.

Common inference rules used in logical systems include:

- Modus Ponens: from P and $P \Rightarrow Q$, infer Q. (Example: 'It is raining' and 'If it is raining then the ground is wet' together let us infer 'the ground is wet'.)
- Modus Tollens: from $\neg Q$ and $P \Rightarrow Q$, infer $\neg P$.
- And-Elimination: from $P \wedge Q$, infer P (and separately infer Q).

- Unit Resolution / Resolution: a single powerful inference rule (used heavily in automated theorem proving) that works on sentences in clausal form and forms the basis of many automated reasoning systems.

Two broad inference strategies exist for rule-based / production systems — chaining forward from known facts towards a goal, or chaining backward from a goal towards known facts — described in detail in the next two sections.

6. Forward Chaining

Forward Chaining is a data-driven inference strategy — reasoning starts from the known facts in the working memory and applies rules whose IF-conditions are satisfied, deriving new facts, which are then added to the working memory; this process repeats, generating more and more new facts, continuing until the goal is reached (i.e., the desired fact appears in the database) or no more rules can fire.

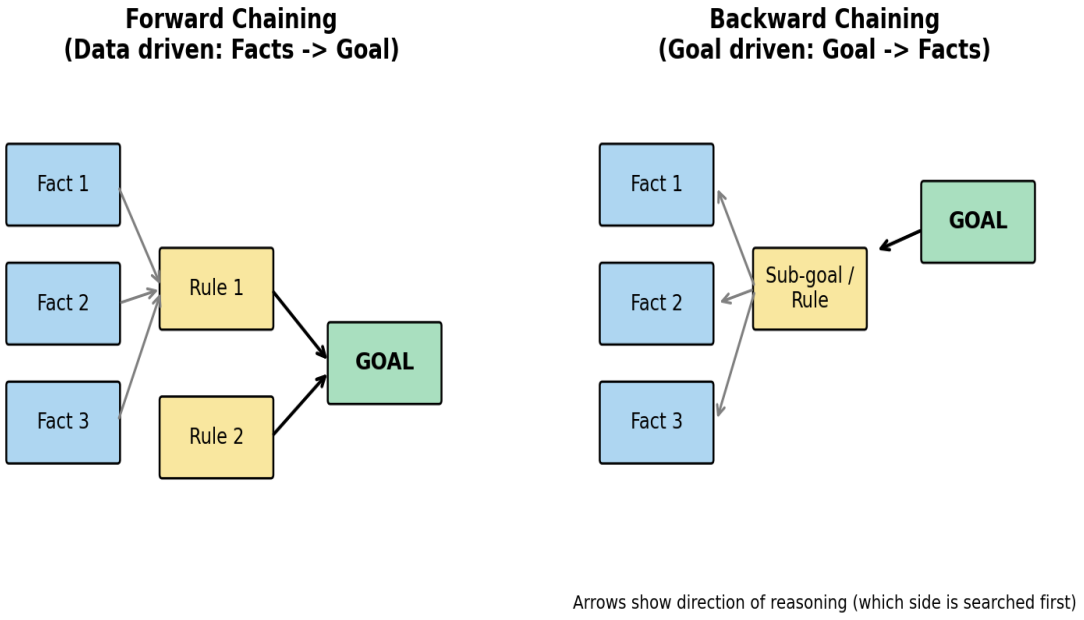


Fig 6.1: Forward Chaining reasons from facts toward the goal (left); Backward Chaining reasons from the goal back to facts (right).

The forward chaining algorithm can be summarised as: (1) collect all facts currently known; (2) find all rules whose IF-part is completely satisfied by these facts; (3) fire those rules, adding their conclusions as new facts (if not already present); (4) repeat steps 2–3 using the enlarged fact set, until either the goal fact appears, or no new facts can be derived.

Example: Given facts 'X is a bird' and rules 'IF X is a bird THEN X can fly' and 'IF X can fly THEN X is not a fish', forward chaining would first apply the rule to derive 'X can fly', and then use this new fact to further derive 'X is not a fish' — arriving step by step at conclusions without initially knowing exactly which conclusion is the goal.

Forward chaining is most suitable when there are many possible conclusions but the number of known initial facts is relatively small, or when the system needs to derive all the consequences of the current situation (e.g., monitoring and alarm systems, where all known sensor readings should be used to generate every relevant

alert). Its main drawback is that it may generate many irrelevant facts that have nothing to do with the actual goal, wasting computation.

7. Backward Chaining

Backward Chaining is a goal-driven inference strategy — reasoning starts from the goal (the hypothesis to be proved or the question to be answered) and works backward, looking for rules whose THEN-part (conclusion) matches the goal. For each such rule, its IF-part (conditions) become new sub-goals, which must themselves be proved (either because they are already known facts, or by recursively finding further rules that conclude them). This continues until all sub-goals are reduced to facts that are already known to be true (or the search fails, meaning the goal cannot be proved).

The backward chaining algorithm can be summarised as: (1) start with the goal to be proved; (2) find a rule whose conclusion (THEN-part) matches the current goal; (3) treat each condition in that rule's IF-part as a new sub-goal; (4) recursively try to prove each sub-goal, either by matching it directly against known facts, or by finding another rule that concludes it; (5) if all sub-goals of some applicable rule succeed, the original goal is proved.

Example: To check the goal 'X is not a fish', backward chaining looks for a rule concluding this — 'IF X can fly THEN X is not a fish' — and sets 'X can fly' as a new sub-goal. It then looks for a rule concluding 'X can fly' — 'IF X is a bird THEN X can fly' — setting 'X is a bird' as the next sub-goal. If 'X is a bird' is already a known fact, the chain of sub-goals succeeds, proving that 'X is not a fish', without ever needing to derive unrelated facts.

Backward chaining is most suitable when there is a single, specific goal to be verified (e.g., diagnostic / expert systems such as MYCIN, where the goal is to confirm 'does the patient have disease D?'), because it only explores rules and facts that are actually relevant to that particular goal, avoiding the wasted effort forward chaining may incur.

7.1 Forward vs Backward Chaining — Comparison

Aspect	Forward Chaining	Backward Chaining
Direction	Facts → Goal (data-driven)	Goal → Facts (goal-driven)
Starts from	Known facts in working memory	The goal / hypothesis to prove
Best suited for	Many possible conclusions; monitoring/ planning systems	A single specific goal; diagnostic/expert systems
Efficiency	Can generate many irrelevant facts	Explores only goal-relevant rules
Example system	Production planning, alarm/monitoring systems	MYCIN-style medical diagnosis

Exam Tip: A very frequent exam question is 'Differentiate between Forward and Backward Chaining' — the table above covers all key points needed for full marks.

UNIT IV — Machine Learning Foundations

This unit introduces the concept of learning in intelligent agents and builds up the foundational algorithms of Machine Learning that form the backbone of modern Artificial Intelligence systems. We move from the general idea of an agent that improves its own performance through experience, to concrete algorithms such as Decision Trees, Support Vector Machines, Artificial Neural Networks and Bayesian Belief Networks. Each topic is explained conceptually, mathematically (where required) and with a worked example so that it can be directly used for examination writing.

1. Learning from Agents (Learning Agents)

Definition: A learning agent is an intelligent agent that is capable of improving its behaviour and performance over time through experience, rather than relying only on a fixed, pre-programmed set of rules.

In classical AI, an agent's behaviour is fixed at design time by the programmer — every possible situation and its correct response must be anticipated in advance. This works only for very simple, fully known environments. Real-world environments, however, are large, dynamic and only partially known in advance. It is impossible for a designer to foresee every situation the agent will encounter. The solution is to build agents that can learn: agents that start with some basic knowledge and then adapt and improve their decision-making as they gather more experience from the environment. Learning allows an agent to operate successfully in environments that were not fully known at design time, and it allows the agent's performance to improve automatically over its lifetime.

Why Build Learning Agents?

- The designer cannot anticipate all possible situations the agent may face.
- The designer cannot anticipate all changes that may occur over time in a dynamic environment.
- Sometimes the designer has no idea how to program a solution manually (e.g., recognising handwritten characters), but the agent itself can be trained using examples.

General Model / Architecture of a Learning Agent

A learning agent can be conceptually divided into four functional components. This architecture is a standard model used to explain how learning is integrated into an agent's decision cycle.

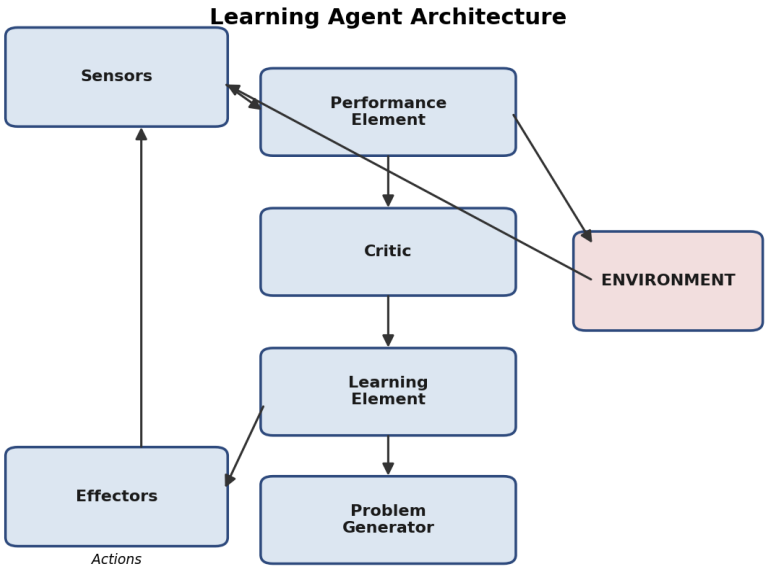


Fig 1.1 — General architecture of a Learning Agent

1. Performance Element: This is what was previously considered the entire agent — it takes in percepts from the sensors and decides on external actions to be performed by the effectors. It is responsible for selecting actions based on the current knowledge it has.
2. Critic: The critic tells the learning element how well the agent is performing with respect to a fixed performance standard. The critic uses an external standard because the percepts themselves do not directly indicate the agent's own success — for example, a chess program may need an external standard to know that checkmating the opponent is good, since the percept sequence alone does not say so.
3. Learning Element: This component is responsible for making improvements to the performance element. It takes knowledge about how the agent has been doing, and determines how the performance element should be modified to do better in the future.
4. Problem Generator: This component is responsible for suggesting actions that will lead to new and informative experiences, even if these actions are not optimal in the short run. This is important because if the agent only ever acts to maximise its immediate performance, it may never discover better long-term strategies — this is often called the exploration vs. exploitation trade-off.

Example

Consider a self-driving taxi agent. The performance element decides steering, acceleration and braking actions. The critic observes outcomes such as sudden braking, passenger complaints, or safe/late arrivals and reports this to the learning element. The learning element then modifies the driving rules (e.g., 'brake earlier when it is raining'). The problem generator may occasionally suggest trying a new, unexplored route purely to learn whether it is faster — even though the agent is not certain it will save time.

2. Inductive Learning

Definition: Inductive learning (also called learning from examples) is a form of learning in which an agent tries to figure out a general function or rule from a given set of input-output example pairs (the training set).

The task is: given a training set of examples of the form (x, f(x)), where x is an input and f(x) is the correct output produced by an unknown target function f, the learner must find (or approximate) a hypothesis function h such that h behaves the same as, or as close as possible to, f — not only on the training examples already seen, but also on new, unseen examples. This ability to correctly handle new inputs is called generalisation.

Key Terminology

- Training Set: The set of example input-output pairs used to construct the hypothesis.
- Hypothesis (h): A candidate function that the learning algorithm proposes as an approximation of the true target function f.
- Hypothesis Space: The set of all hypotheses that the learning algorithm is allowed to consider (e.g., all straight lines, all decision trees of a certain depth).
- Consistent Hypothesis: A hypothesis h is said to be consistent with the training data if it agrees with the value of f for every example in the training set.
- Generalisation / Test Set: The ability of the learned hypothesis to correctly predict outputs for inputs that were not part of the training set; measured using a separate test set.

Principle of Ockham's Razor

When several hypotheses are equally consistent with the training data, inductive learning prefers the simplest hypothesis that fits the data. This principle, called Ockham's Razor, is used because simpler hypotheses are less likely to just be memorising noise in the data, and are more likely to generalise well to new data.

Example

Suppose we are given the following example pairs of a function: f(1) = 2, f(2) = 4, f(3) = 6, f(4) = 8. A learner using inductive learning may propose the hypothesis h(x) = 2x. This hypothesis is consistent with all the given examples, and because it is a simple linear rule, by Ockham's Razor it would be preferred over a more complicated hypothesis (such as a high-degree polynomial) that also happens to fit the same four points exactly.

A very common way to visualise inductive learning for two-class data is as finding a curve or boundary that best separates the plotted example points, such that it can correctly classify new points that fall on either side of that boundary.

3. Types of Machine Learning

Machine Learning algorithms are broadly classified into the following categories, based on the nature of the training data (whether it is labelled) and the kind of feedback the learner receives from the environment.

Type	Description	Example
Supervised Learning	The agent learns a mapping from inputs to outputs using a training set of correctly labelled input-output pairs.	Spam email detection; predicting house prices
Unsupervised Learning	The agent learns patterns, structure or groupings in the input data without any labelled outputs being provided.	Customer segmentation; clustering documents

Type	Description	Example
Semi-Supervised Learning	The agent learns from a training set that contains a small amount of labelled data along with a large amount of unlabelled data.	Labelling a few images and using many unlabelled ones to improve accuracy
Reinforcement Learning	The agent learns a policy of actions by interacting with an environment and receiving rewards or punishments (reinforcement) rather than being told the correct action directly.	A game-playing agent that learns by winning/ losing games

Table 3.1 — Types of Machine Learning

Unit IV of this course primarily focuses on Supervised Learning techniques (Decision Trees, SVM, Neural Networks, Bayesian Networks), while Unit V focuses on Unsupervised Learning (Clustering methods).

4. Supervised Learning

Definition: Supervised learning is a type of machine learning in which the algorithm is trained on a labelled dataset — that is, a dataset in which every training example is paired with the correct output (label). The goal is to learn a mapping function from inputs to outputs so that the model can predict the output for new,

The word 'supervised' comes from the idea that a 'teacher' (the labelled dataset) supervises the learning process by providing the correct answer for every training example, and the algorithm's job is to adjust itself so that its predicted output matches the correct output as closely as possible, minimising the prediction error.

Steps in Supervised Learning

5. Collect a labelled training dataset consisting of input-output pairs.
6. Choose a suitable hypothesis representation / model (e.g., decision tree, neural network, SVM).
7. Train the model: adjust its internal parameters so that its predictions match the training labels as closely as possible, typically by minimising an error/loss function.
8. Validate/test the trained model on a separate set of examples (the test set) that were not used during training, to check how well it generalises.
9. Use the trained model to make predictions on completely new, unseen data.

Categories of Supervised Learning Problems

- Classification: The output variable is a discrete category/class label. Example: classifying an email as 'Spam' or 'Not Spam'.
- Regression: The output variable is a continuous numeric value. Example: predicting the price of a house based on its area, location and number of rooms.

Example

Suppose we want to build a system that predicts whether a student will pass or fail an exam based on the number of hours studied. We collect past data of (hours studied, pass/fail) pairs and train a supervised learning

model on this labelled data. Once trained, the model can predict pass/fail for a new student given only their study hours. This is a classification problem, since the output ('pass' or 'fail') is a discrete category.

5. Learning Decision Trees

Definition: A Decision Tree is a supervised learning model represented as a tree structure, where each internal node tests the value of a particular attribute, each branch represents an outcome of that test, and each leaf node represents a final decision or class label.

Decision tree learning is one of the most widely used and easy-to-interpret methods of inductive learning. Given a set of labelled training examples, the algorithm builds a tree that can be used to classify new, unseen examples by starting at the root node, testing the attribute at that node, following the branch corresponding to the example's value for that attribute, and repeating this process until a leaf node (a decision) is reached.

Structure of a Decision Tree

- Root Node: Represents the entire dataset and the first (most important) attribute test.
- Internal (Decision) Node: Represents a test on a particular attribute.
- Branch: Represents the outcome of a test, leading to the next node.
- Leaf Node: Represents the final classification/decision (class label).

The ID3 Algorithm (Building a Decision Tree)

The most classical algorithm for constructing a decision tree from training data is ID3 (Iterative Dichotomiser 3). The central idea of ID3 is to choose, at every step, the attribute that is most useful for classifying the examples — that is, the attribute that best splits the training examples into groups that are as 'pure' (uniform in class label) as possible. This 'usefulness' is measured mathematically using the concepts of Entropy and Information Gain.

Entropy

Entropy is a measure, borrowed from information theory, of the amount of uncertainty or impurity/disorder present in a set of examples. If a set S contains only examples of a single class, its entropy is 0 (no uncertainty at all). If a set S is evenly split between two classes, its entropy is at its maximum, 1 (highest uncertainty). For a set S with a Boolean (two-class) classification, where $p\oplus$ is the proportion of positive examples and $p\ominus$ is the proportion of negative examples:

$$\textbf{Entropy}(S) = -\, p\oplus \log_2(p\oplus) - p\ominus \log_2(p\ominus)$$

Information Gain

Information Gain measures the expected reduction in entropy that results from splitting (partitioning) the training examples according to a particular attribute A. The attribute with the highest information gain is chosen as the decision node, because it does the best job of separating the training examples according to the target classification.

$$\textbf{Gain}(S, A) = \textbf{Entropy}(S) - \sum \left(|S_v| / |S| \right) \times \textbf{Entropy}(S_v)$$

where S_v is the subset of S for which attribute A has value v, and the summation is taken over every possible value v of attribute A.

Worked Example: 'Should We Play Tennis?'

Consider the classic training set below, where we want to decide, based on weather conditions, whether a game of tennis will be played (Yes/No).

Day	Outlook	Humidity	Wind	Play Tennis?
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No

Table 5.1 — Sample training data

Step 1: Out of 8 examples, 4 are 'Yes' and 4 are 'No'. So the initial Entropy(S) = $-(4/8)\log_2(4/8) - (4/8)\log_2(4/8) = 1$ (maximum uncertainty, since the classes are evenly split).

Step 2: We calculate the Information Gain for each candidate attribute (Outlook, Humidity, Wind) by computing the weighted entropy of the subsets formed after splitting on that attribute, and subtracting it from 1. In this example, splitting on 'Outlook' produces the purest subsets — all 'Overcast' examples are 'Yes', which gives Outlook the highest information gain.

Step 3: Since Outlook gives the highest information gain, it becomes the root node. The examples are partitioned into three branches: Sunny, Overcast and Rain. The 'Overcast' branch becomes a pure leaf ('Yes') immediately, since all overcast examples say 'Yes'. For the 'Sunny' and 'Rain' branches, which still contain a mix of Yes/No, the algorithm recursively repeats the same process (choosing the next best attribute — Humidity for Sunny, Wind for Rain) until every branch ends in a pure leaf node.

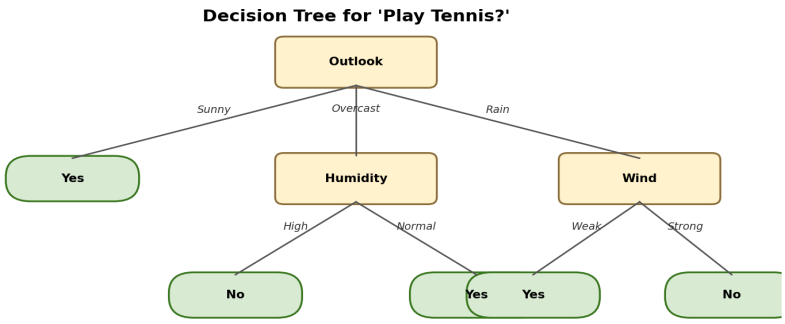


Fig 5.1 — Resulting decision tree for the 'Play Tennis' example

This resulting tree can now be used to classify a brand-new day: to predict whether tennis will be played, we simply start at the root, check the actual Outlook value for that day, and follow the tree down to a leaf to get the prediction.

Advantages of Decision Trees

- Easy to understand, interpret and visualise, even by non-experts.
- Requires little data preparation (no need for feature scaling/normalisation).
- Can handle both categorical and numerical data.

Disadvantages of Decision Trees

- Prone to overfitting, especially when the tree is allowed to grow very deep, causing it to model noise in the training data.
- Small changes in data can result in a completely different tree being generated (instability).
- Can create biased trees if some classes dominate the dataset.

6. Support Vector Machines (SVM)

Definition: A Support Vector Machine is a supervised learning algorithm that classifies data by finding the optimal hyperplane that best separates the data points of different classes, such that the margin (distance) between the hyperplane and the nearest data points of each class is maximised.

Support Vector Machines are considered one of the most powerful 'out-of-the-box' classifiers in machine learning, particularly effective for classification problems with a clear margin of separation between classes and for high-dimensional data.

Key Concepts

- Hyperplane: A decision boundary that separates different classes of data. In a 2-dimensional space this is simply a line; in 3 dimensions it is a plane; in higher dimensions it is called a hyperplane.
- Margin: The distance between the hyperplane and the nearest data point from either class. SVM tries to find the hyperplane that has the maximum possible margin — this is why it is also called the Maximum Margin Classifier.
- Support Vectors: The data points that lie closest to the hyperplane. These points are the most critical elements of the dataset because they are the ones that actually determine the position and orientation of the hyperplane — removing any non-support-vector point would not change the hyperplane at all, but moving a support vector would.
- Kernel Trick: When data cannot be separated by a straight line/hyperplane in its original space (i.e., it is not linearly separable), SVM uses a mathematical function called a kernel to transform/project the data into a higher-dimensional space in which it does become linearly separable. Common kernels include Linear, Polynomial and Radial Basis Function (RBF).

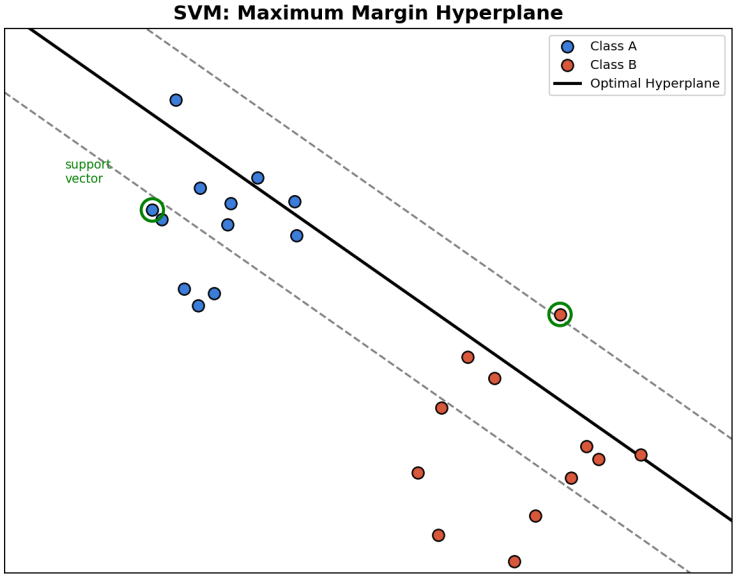


Fig 6.1 — Maximum margin hyperplane separating two classes, with support vectors circled

Why Maximise the Margin?

A hyperplane that is very close to the training points of one class is considered 'unsafe', because even a small amount of noise or a new, slightly different test point could easily be misclassified. By choosing the hyperplane with the maximum margin, SVM builds in the largest possible safety buffer on both sides, which generally leads to better generalisation on unseen data compared to a hyperplane that just barely separates the classes.

Soft Margin and the Regularisation Parameter (C)

In real-world datasets, the classes are rarely perfectly separable — a few data points may lie on the wrong side of the boundary due to noise or overlap. SVM handles this using the concept of a soft margin, which allows a limited number of misclassifications, controlled by a regularisation parameter usually denoted C. A large value of C forces the model to classify all training points correctly (risking overfitting), whereas a small value of C allows a wider margin at the cost of a few misclassifications (which usually generalises better).

Example

Suppose we want to classify emails as spam or not-spam, using two features: the number of times the word 'free' appears, and the number of exclamation marks used. If we plot each email as a point on a 2-D graph using these two feature values, spam emails tend to cluster in one region (high 'free' count, many exclamation marks) while genuine emails cluster elsewhere. SVM finds the straight line (hyperplane) that best divides these two clusters with the widest possible gap, and any new email is classified as spam or not-spam depending on which side of that line it falls.

Advantages of SVM

- Effective in high-dimensional spaces, even when the number of features exceeds the number of samples.
- Memory efficient, since it uses only a subset of training points (the support vectors) in the decision function.
- Versatile — different kernel functions can be applied for different types of decision boundaries.

Disadvantages of SVM

- Not very efficient on very large datasets, since training time can be high.
- Performance depends heavily on the correct choice of kernel and its parameters.
- Does not directly provide probability estimates for class membership.

7. Neural Networks (Introduction) and Belief Networks

Artificial Neural Networks (ANNs) are computing systems designed to loosely mimic the way biological brains process information. A biological brain is made up of billions of interconnected cells called neurons; each neuron receives electrical/chemical signals from other neurons through connections called synapses, and 'fires' (sends its own signal onward) only if the combined incoming signal crosses a certain threshold. Artificial Neural Networks borrow this same basic idea — a network of simple processing units (artificial neurons), connected together, that can collectively learn to perform complex tasks such as pattern recognition, classification and prediction.

Biological Neuron vs Artificial Neuron

Biological Neuron	Artificial Neuron (Analogy)
Dendrites (receive signals)	Inputs (x1, x2, …, xn)
Synaptic strength	Weights (w1, w2, …, wn)
Cell body (sums signals)	Summation function (Σ)
Firing threshold	Activation function
Axon (sends output signal)	Output (y)

Table 7.1 — Biological neuron and its artificial analogy

The term Belief Network generally refers to Bayesian Belief Networks, which represent a different, probabilistic approach to knowledge and learning (covered in detail in Topic 10 below), as opposed to Neural Networks, which learn numeric weights through training examples. Both are important sub-areas used for learning and reasoning under uncertainty.

8. The Perceptron

Definition: A Perceptron is the simplest form of an artificial neural network — a single artificial neuron that takes several binary or real-valued inputs, computes a weighted sum of these inputs, and produces a single binary output based on whether this sum exceeds a certain threshold.

The perceptron was proposed by Frank Rosenblatt in 1958 and is historically the earliest trainable neural network model. It is primarily used for binary classification problems where the data is linearly separable — that is, the two classes can be separated by drawing a single straight line (in 2-D) or a hyperplane (in higher dimensions).

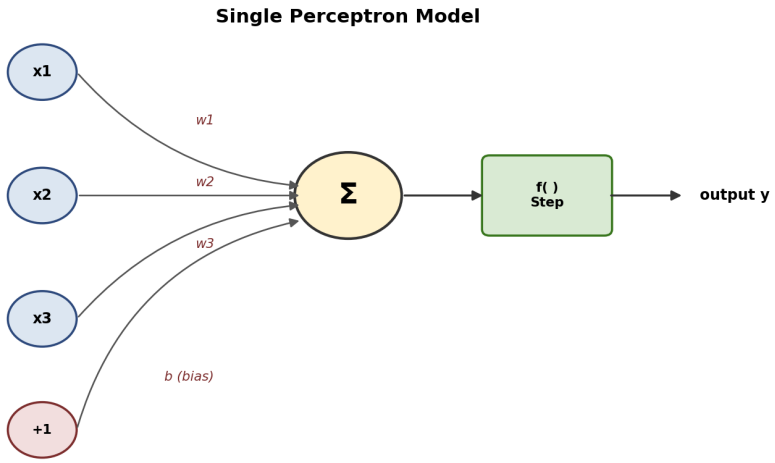


Fig 8.1 — Structure of a single perceptron

Working of a Perceptron

Each input x_i is multiplied by its corresponding weight w_i, and all these weighted inputs are summed together, along with an extra term called the bias (b), which allows the decision boundary to be shifted away from the

origin. This weighted sum is then passed through an activation function — in the original perceptron, this is a simple step function that outputs 1 if the sum is above a threshold, and 0 (or −1) otherwise.

Weighted Sum: $z = (w1 \cdot x1 + w2 \cdot x2 + \ldots + wn \cdot xn) + b$

Output: $y = 1$ if $z \geq 0$, $y = 0$ if $z < 0$

The Perceptron Learning Rule

The perceptron learns by adjusting its weights whenever it makes a mistake on a training example. If the predicted output $\hat{y}$ does not match the true/target output t for a training example, the weights are updated as follows, where η (eta) is a small positive number called the learning rate:

$w_i(new) = w_i(old) + \eta \times (t - \hat{y}) \times x_i$

This rule has a simple intuition: if the perceptron predicted 0 but the correct answer was 1 ($t - \hat{y} = 1$), the weights are increased in the direction of the inputs that were 'on', making the perceptron more likely to output 1 next time for a similar input. If it predicted 1 but the correct answer was 0, the weights are decreased. This process is repeated over all training examples, for many passes (epochs), until the perceptron classifies all training examples correctly (or a maximum number of iterations is reached).

Limitation of the Perceptron

A single perceptron can only learn to correctly classify data that is linearly separable. A classic example of its limitation is the XOR (exclusive-OR) problem: there is no single straight line that can separate the (0,0)/(1,1) points (output 0) from the (0,1)/(1,0) points (output 1) in the XOR truth table. This limitation, famously pointed out by Minsky and Papert in 1969, led to a temporary decline of interest in neural networks, and was eventually solved by using multiple layers of perceptrons — the Multi-layer Feed Forward Network described next.

9. Multi-Layer Feed Forward Networks (Multi-Layer Perceptron)

Definition: A Multi-Layer Feed Forward Network (also called a Multi-Layer Perceptron, MLP) is a neural network consisting of more than one layer of neurons — an input layer, one or more hidden layers, and an output layer — in which information flows strictly in one direction, from input to output, with no cycles or

By stacking multiple layers of neurons, with non-linear activation functions at each neuron, a multi-layer network is able to represent much more complex, non-linear decision boundaries than a single perceptron — including problems like XOR that a single perceptron cannot solve.

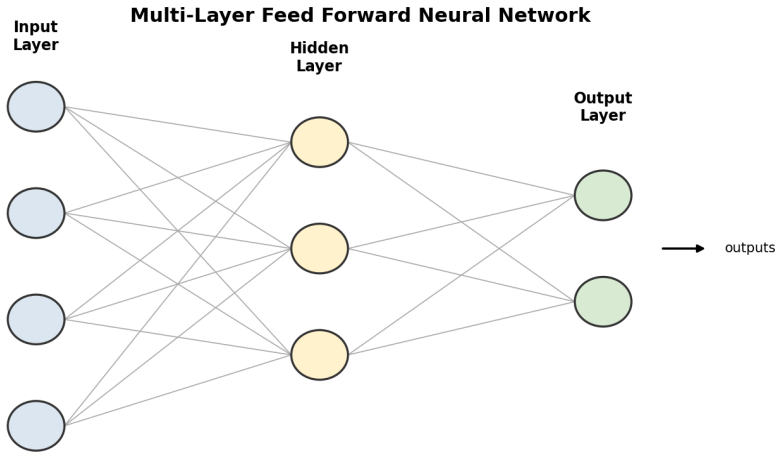


Fig 9.1 — A multi-layer feed forward neural network with one hidden layer

Layers of an MLP

- Input Layer: Receives the raw feature values of the example; performs no computation, simply passes values forward.
- Hidden Layer(s): One or more intermediate layers where the actual non-linear computation happens. Each hidden neuron computes a weighted sum of its inputs and passes it through a non-linear activation function (e.g., Sigmoid, Tanh, ReLU).
- Output Layer: Produces the final prediction of the network (e.g., a class label or a numeric value).

Training an MLP: Backpropagation

Because an MLP has hidden layers whose 'correct' output is not directly known from the training data (only the final output layer's target is known), a special training algorithm called Backpropagation (short for 'backward propagation of errors') is used. Backpropagation works in two phases, repeated for every training example (or batch), over many epochs:

10. Forward Pass: The input is passed forward through the network layer by layer, using the current weights, to produce a predicted output.
11. Backward Pass: The error (difference between predicted output and the true target) is calculated at the output layer, and then propagated backward through the network, layer by layer, using the chain rule of calculus, to compute how much each individual weight contributed to the error.
12. Weight Update: Every weight in the network is then adjusted slightly, in the direction that reduces the overall error, using an optimisation method such as Gradient Descent.

This process is repeated for many epochs until the network's overall error on the training set becomes acceptably small, or stops improving further.

Example

Consider handwritten digit recognition. The input layer receives the pixel intensity values of a digit image. The hidden layers progressively learn to detect useful intermediate features — for instance, early hidden neurons might learn to detect edges and curves, while later hidden neurons combine these into shapes resembling parts of digits. The output layer finally has 10 neurons (one for each digit 0–9), and the neuron with the highest output value is taken as the network's predicted digit.

10. Bayesian Belief Networks

Definition: A Bayesian Belief Network (BBN), also called a Bayesian Network or a Belief Network, is a graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG), together with a Conditional Probability Table (CPT) attached to every node.

Bayesian Networks provide a way to represent uncertain knowledge compactly and to perform reasoning under uncertainty. Instead of storing the full joint probability distribution over all variables (which grows exponentially with the number of variables and quickly becomes infeasible to store or compute), a Bayesian Network exploits the fact that most variables directly depend on only a few other variables, and represents only these local dependencies explicitly.

Structure of a Bayesian Network

- Nodes: Each node represents a random variable, which may be discrete (e.g., True/False) or continuous.
- Directed Edges: An arrow from node A to node B indicates that A has a direct causal or influential effect on B — B is said to be conditionally dependent on A. There must be no directed cycles in the graph (hence, Directed Acyclic Graph).
- Conditional Probability Table (CPT): Every node has an attached CPT that quantifies the effect of its parent nodes on it. For a node with no parents, the CPT simply stores its prior probability.

Worked Example: The Burglary–Alarm Network

This is a classic textbook example. Suppose you have installed a burglar alarm at your house. The alarm is fairly reliable at detecting a burglary, but it can also be triggered occasionally by a minor earthquake. You have two neighbours, John and Mary, who have both promised to call you at work if they hear the alarm — but John sometimes confuses the alarm with a phone ringing and calls anyway, and Mary sometimes misses the alarm because she likes loud music.

This scenario can be represented using five random variables — Burglary, Earthquake, Alarm, JohnCalls and MaryCalls — connected as shown below:

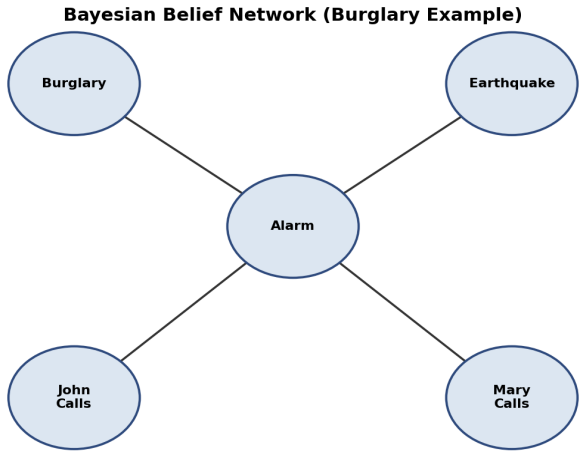


Fig 10.1 — Bayesian Belief Network for the burglary example

Here, Burglary and Earthquake are root nodes (no parents), each with its own prior probability of occurring. Alarm depends on both Burglary and Earthquake (it can be triggered by either or both), so its CPT specifies P(Alarm | Burglary, Earthquake) for all four combinations of the two parent values. Finally, JohnCalls and MaryCalls each depend only on Alarm (not directly on Burglary or Earthquake), reflecting the fact that John and Mary only react to hearing the alarm, not to the burglary or earthquake directly.

Illustrative CPT for the Alarm node

Burglary	Earthquake	P(Alarm = True)
True	True	0.95
True	False	0.94
False	True	0.29
False	False	0.001

Table 10.1 — Sample Conditional Probability Table for the Alarm node

Computing Joint Probability

One of the most useful properties of a Bayesian Network is that it allows the full joint probability distribution over all its variables to be computed as a simple product of the individual CPT entries, using the chain rule for Bayesian Networks:

$P(X1, X2, \ldots, Xn) = \Pi\ P(X_i \mid Parents(X_i))$

This means we never need to store the entire joint distribution table (which, for 5 Boolean variables, would need $2^5 = 32$ entries); instead we only need the much smaller set of local CPTs, and the full joint probability of any specific combination of events can be calculated on demand by simply multiplying together the relevant conditional probabilities from the network.

Applications of Bayesian Belief Networks

- Medical diagnosis — reasoning about diseases and their symptoms under uncertainty.
- Spam filtering and document classification.
- Risk analysis and decision support systems.
- Fault diagnosis in industrial and mechanical systems.

Advantages

- Compactly represents uncertain knowledge, avoiding the need to store a huge joint probability table.
- Explicitly shows the causal/dependency structure between variables, making the model easy to interpret.
- Supports both forward (predictive) reasoning and backward (diagnostic) reasoning.

UNIT V — Unsupervised Learning

While Unit IV dealt with supervised learning, where the training data comes with correct labels/answers, this unit deals with Unsupervised Learning, where the algorithm is given only raw, unlabelled data and must discover hidden structure, patterns or natural groupings within it on its own. The main focus of this unit is Clustering — the task of grouping similar data points together — covering K-Means Clustering, Hierarchical Clustering (Agglomerative and Divisive) and Fuzzy Clustering.

1. Unsupervised Learning

Definition: Unsupervised learning is a type of machine learning in which the algorithm is given a dataset that has no labelled outputs, and it must find natural patterns, groupings or structure within the data purely based on the similarities and differences among the data points themselves.

Unlike supervised learning, where a 'teacher' provides the correct answer for every training example, unsupervised learning has no such teacher — the algorithm must explore the data and organise it in a meaningful way based only on the inherent structure present in the features. This is very useful in real-world situations where labelling data is expensive, time-consuming, or simply not possible.

Common Tasks in Unsupervised Learning

- Clustering: Grouping data points into clusters such that points within the same cluster are more similar to each other than to points in other clusters.
- Dimensionality Reduction: Reducing the number of features in the data while preserving as much important information as possible (e.g., Principal Component Analysis).
- Association Rule Learning: Discovering interesting relationships/associations between variables in large datasets (e.g., market basket analysis — 'customers who buy bread also tend to buy butter').

Supervised vs Unsupervised Learning

Aspect	Supervised Learning	Unsupervised Learning
Training data	Labelled (input-output pairs given)	Unlabelled (only inputs given)
Goal	Learn a mapping to predict output for new inputs	Discover hidden structure/groupings in data
Feedback	Direct — correct answer is known	No direct feedback / correct answer
Example algorithms	Decision Trees, SVM, Neural Networks	K-Means, Hierarchical Clustering, Fuzzy C-Means
Typical use	Classification, Regression	Clustering, Pattern discovery

Table 1.1 — Supervised vs Unsupervised Learning

Example

A retail company has data about the purchasing behaviour of thousands of customers, but no predefined labels about which 'type' of customer each one is. Using unsupervised learning (clustering), the company can automatically group customers into segments such as 'bargain hunters', 'brand loyal customers' and 'occasional

buyers', purely based on similarities in their purchase patterns, without ever having been told in advance what these groups should be.

2. K-Means Clustering

Definition: K-Means is a partition-based unsupervised clustering algorithm that divides a given dataset into a pre-specified number, K, of non-overlapping clusters, such that each data point belongs to the cluster with the nearest mean (centroid).

K-Means is one of the simplest and most widely used clustering algorithms because of its speed and simplicity. It works iteratively, alternating between assigning points to the nearest cluster centre and then recalculating the cluster centres, until the assignments stop changing.

Steps of the K-Means Algorithm

13. Choose the number of clusters, K, that the data should be divided into.
14. Randomly initialise K points in the feature space as the initial cluster centroids.
15. Assignment Step: For every data point in the dataset, calculate its distance (usually Euclidean distance) to each of the K centroids, and assign the point to the cluster whose centroid is nearest.
16. Update Step: Recalculate each cluster's centroid by taking the mean (average) of all the data points currently assigned to that cluster.
17. Repeat the Assignment Step and Update Step until the centroids no longer change significantly (convergence), i.e., points stop switching clusters.

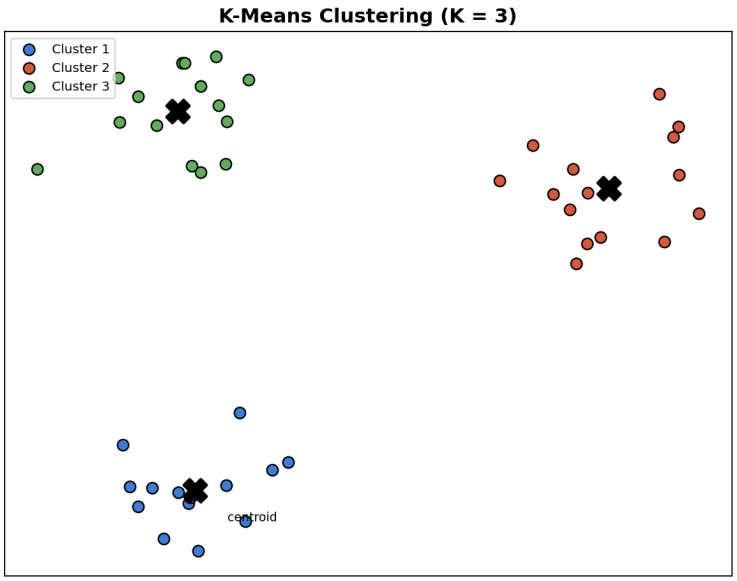


Fig 2.1 — Data points partitioned into K = 3 clusters, with centroids marked ✕

Distance Formula Used

The Euclidean distance between a data point x = (x1, x2, …, xn) and a centroid c = (c1, c2, …, cn) is calculated as:

distance(x, c) = √((x1−c1)² + (x2−c2)² + … + (xn−cn)²)

Worked Example

Suppose we have six students' marks in two subjects and we want to group them into K = 2 clusters ('weak performers' and 'strong performers'). We first randomly choose two students' mark-pairs as the initial centroids. In the assignment step, every student is assigned to whichever of the two centroids their marks are closest to. In the update step, we recompute each cluster's centroid as the average marks of the students currently in that cluster. We repeat this assignment-and-update cycle; after a few iterations the centroids stabilise, and the six students end up divided into two clear groups — one cluster containing students with generally lower marks and the other containing students with generally higher marks.

Choosing the Value of K: The Elbow Method

A common practical problem with K-Means is that the number of clusters K must be specified in advance. The Elbow Method is a popular technique to choose a good value of K: we run K-Means for a range of K values, and for each K, we compute the total within-cluster sum of squared distances (a measure of how compact the clusters are). As K increases, this value keeps decreasing, but after a certain point the rate of decrease slows down sharply, forming an 'elbow' shape on the graph. The value of K at this elbow point is usually chosen as the best trade-off between accuracy and simplicity.

Advantages

- Simple to understand and computationally fast, even for fairly large datasets.
- Guaranteed to converge (though not always to the global optimum).

Disadvantages

- The number of clusters K must be specified in advance.
- Sensitive to the initial random placement of centroids — poor initialisation can lead to poor clustering.
- Works best only when clusters are roughly spherical/globular in shape and of similar size; performs poorly on irregularly shaped clusters.
- Sensitive to outliers, since they can significantly distort the mean of a cluster.

3. Hierarchical Clustering

Definition: Hierarchical Clustering is an unsupervised clustering method that builds a hierarchy (a tree-like structure) of nested clusters, rather than producing a single fixed set of K clusters, by progressively merging or splitting clusters based on their similarity.

Unlike K-Means, hierarchical clustering does not require the number of clusters to be specified in advance. Instead, it produces a complete tree of clusters, called a Dendrogram, and the user can 'cut' this tree at any desired level to obtain any number of clusters after the algorithm has already run.

The Dendrogram

A dendrogram is a tree-like diagram that records the sequence of merges (or splits) made during hierarchical clustering, along with the distance (dissimilarity) at which each merge/split happened. The height at which two clusters are joined in the dendrogram represents how dissimilar those two clusters were. To obtain a specific number of clusters, we simply draw a horizontal line across the dendrogram at a chosen height — the number of vertical lines this horizontal cut crosses gives the number of clusters.

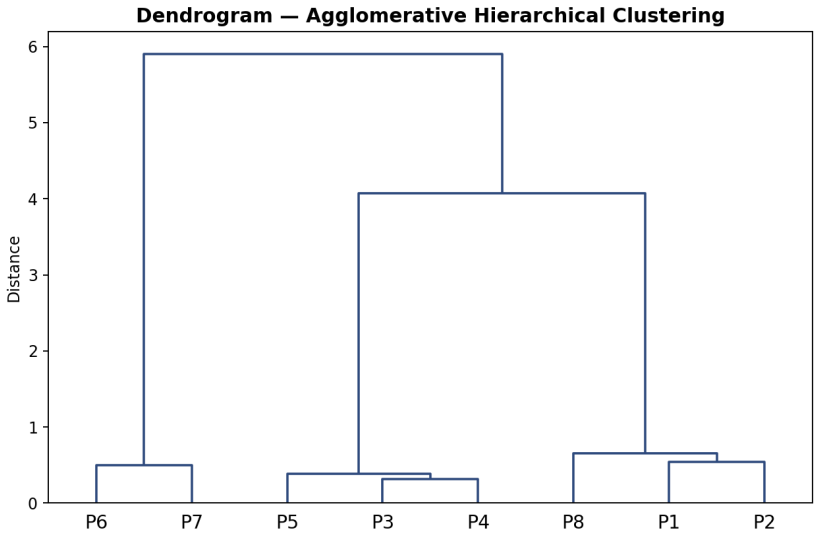


Fig 3.1 — A dendrogram showing the hierarchy of merges in agglomerative clustering

Hierarchical clustering can be performed using two fundamentally opposite strategies: Agglomerative (bottom-up) and Divisive (top-down), explained in detail below.

4. Agglomerative and Divisive Clustering

(a) Agglomerative Clustering (Bottom-Up Approach)

Definition: Agglomerative clustering is a 'bottom-up' hierarchical clustering approach that starts by treating every single data point as its own individual cluster, and then repeatedly merges the two closest (most similar) clusters together, one pair at a time, until only a single cluster containing all the data points remains.

Steps of Agglomerative Clustering

18. Start with every data point as a separate, individual cluster (so N data points give N initial clusters).

19. Compute the distance/similarity between every pair of clusters.
20. Merge the two clusters that are closest to (most similar to) each other into a single new cluster.
21. Update the distance matrix to reflect the distance between this newly formed cluster and all remaining clusters.
22. Repeat steps 2–4 until only one cluster remains, containing all data points; record every merge (and the distance at which it happened) to build the dendrogram.

(b) Divisive Clustering (Top-Down Approach)

Definition: Divisive clustering is a 'top-down' hierarchical clustering approach that starts with all data points in a single large cluster, and then repeatedly splits the most heterogeneous (least similar/most spread out) cluster into two smaller clusters, continuing until every data point is in its own individual cluster.

Divisive clustering is essentially the exact reverse process of agglomerative clustering. It is conceptually simple but computationally more expensive in practice, since deciding the best way to split a cluster into two requires considering a very large number of possible splits (there are 2^(n−1) − 1 ways to split a cluster of n points into two non-empty groups), which is why agglomerative clustering is used far more commonly in practice.

Aspect	Agglomerative (Bottom-Up)	Divisive (Top-Down)
Starting point	Each point is its own cluster	All points in one single cluster
Process	Repeatedly merges closest pairs of clusters	Repeatedly splits the most dissimilar cluster
Ending point	Ends with one large cluster	Ends with each point as its own cluster
Computational cost	Less expensive; more commonly used	More expensive; less commonly used

Table 4.1 — Agglomerative vs Divisive Clustering

Linkage Methods (used to measure distance between clusters)

Whether merging (agglomerative) or splitting (divisive), we need a way to measure the 'distance' between two clusters that each contain multiple points, not just single points. This is done using a linkage criterion:

- Single Linkage: The distance between two clusters is taken as the minimum distance between any single point in the first cluster and any single point in the second cluster.
- Complete Linkage: The distance between two clusters is taken as the maximum distance between any point in the first cluster and any point in the second cluster.
- Average Linkage: The distance between two clusters is taken as the average of the distances between every pair of points, one from each cluster.
- Centroid Linkage: The distance between two clusters is taken as the distance between the centroids (mean points) of the two clusters.

Example

Suppose we have five cities and we want to group them hierarchically based on the road distance between them. Agglomerative clustering would start by treating each city as its own cluster, then merge the two closest cities

into a group, then find the next closest pair of clusters (which could be a city and the group, or two other cities) and merge them, and so on, until all five cities are combined into a single cluster — with every merge step recorded in a dendrogram that can then be 'cut' at any distance threshold to get the desired number of city groups.

5. Fuzzy Clustering

Definition: Fuzzy Clustering is a clustering technique in which each data point can belong to more than one cluster simultaneously, with a degree of membership (a value between 0 and 1) indicating how strongly it belongs to each cluster, rather than being forced to belong to exactly one cluster as in K-Means and

Both K-Means and Hierarchical Clustering are examples of 'hard' or 'crisp' clustering, where every data point is assigned to exactly one cluster, with full (100%) membership in that cluster and 0% membership in every other cluster. In many real-world situations, however, data points do not naturally belong entirely to a single group — they may share characteristics of multiple groups at once. Fuzzy clustering handles this more realistically by allowing partial, overlapping membership.

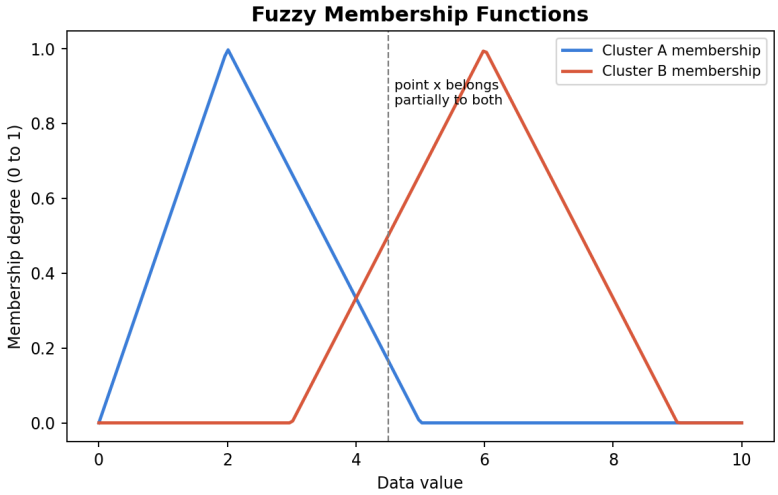


Fig 5.1 — Fuzzy membership functions: a point can partially belong to two clusters at once

Fuzzy C-Means (FCM) Algorithm

The most widely used fuzzy clustering algorithm is Fuzzy C-Means (FCM), which is a fuzzy generalisation of the K-Means algorithm. Instead of a hard 0/1 assignment, every data point x_i is given a membership value u_{ij} (between 0 and 1) for every cluster j, representing the degree to which that point belongs to that cluster. For any given point, the sum of its membership values across all clusters must add up to 1.

Steps of Fuzzy C-Means

23. Choose the number of clusters C, and randomly initialise the membership value of every data point for every cluster.
24. Compute the centroid (fuzzy mean) of every cluster, where each data point's contribution to the centroid is weighted by its degree of membership in that cluster.

25. Update the membership values of every data point for every cluster, based on its distance to each of the newly computed cluster centroids — points closer to a centroid get a higher membership value for that cluster.
26. Repeat the centroid-update and membership-update steps until the membership values converge (stop changing significantly between iterations).

Example

Consider classifying a set of fruit images purely by colour, where we have two clusters: 'ripe' (red/yellow) and 'unripe' (green). A fruit image that is mostly yellow with a slight green tinge cannot be said to belong 100% to either cluster — fuzzy clustering would assign it something like 0.7 membership to 'ripe' and 0.3 membership to 'unripe', which more realistically reflects its actual, in-between appearance, compared to a hard clustering method that would be forced to put it entirely into just one of the two categories.

Advantages of Fuzzy Clustering

- More realistic for data with overlapping or ambiguous groups, where hard boundaries do not really exist.
- Provides richer information — the membership values themselves indicate how confidently a point belongs to a cluster.

Disadvantages of Fuzzy Clustering

- More computationally expensive than K-Means, since membership values for every point-cluster pair must be maintained and updated.
- Choosing the right number of clusters, and interpreting overlapping memberships, can be more complex.

Summary — Quick Revision Table (Unit V)

Method	Approach	Number of clusters	Membership type
K-Means	Partition-based, iterative centroid update	Must be specified (K) in advance	Hard (each point in exactly one cluster)
Agglomerative	Bottom-up merging of clusters	Decided later by cutting dendrogram	Hard
Divisive	Top-down splitting of clusters	Decided later by cutting dendrogram	Hard
Fuzzy C-Means	Partition-based, iterative, weighted	Must be specified (C) in advance	Soft / Fuzzy (partial membership in multiple clusters)